

Object Oriented Programming: Principles

OOP stands for Object-Oriented Programming. OOP is a programming paradigm in which every program follows the concept of object. In other words, OOP is a way of writing programs based on the object concept.

The object-oriented programming paradigm has the following core concepts.

- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- **Abstraction**

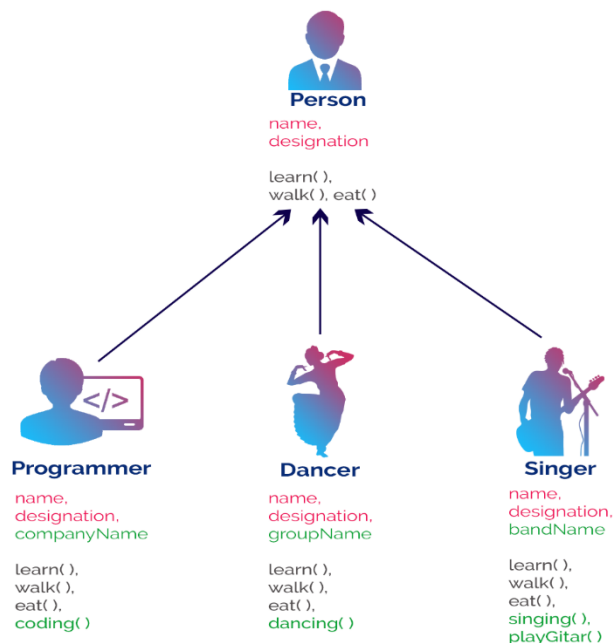
Encapsulation = Data + Code



Class = Variables + Methods

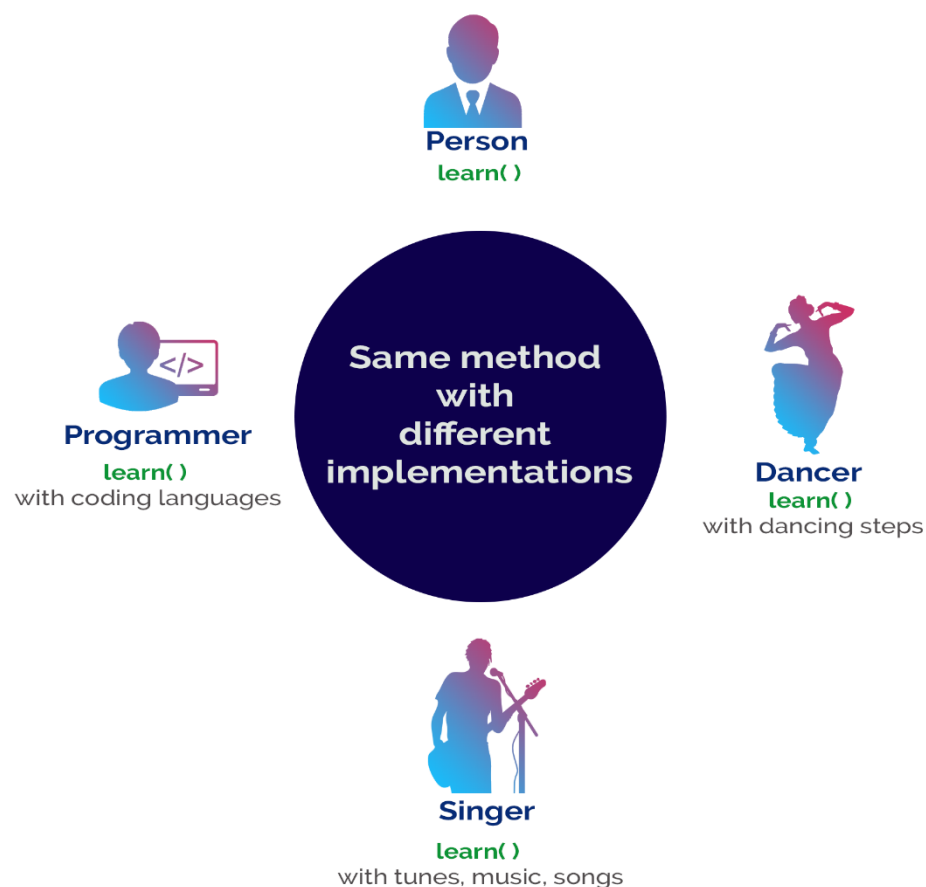
www.btechsmartclass.com

- Encapsulation is the process of combining data and code into a single unit (object / class). In OOP, every object is associated with its data and code. In programming, data is defined as variables and code is defined as methods. The java programming language uses the class concept to implement encapsulation.
- **Inheritance**



www.btechsmartclass.com

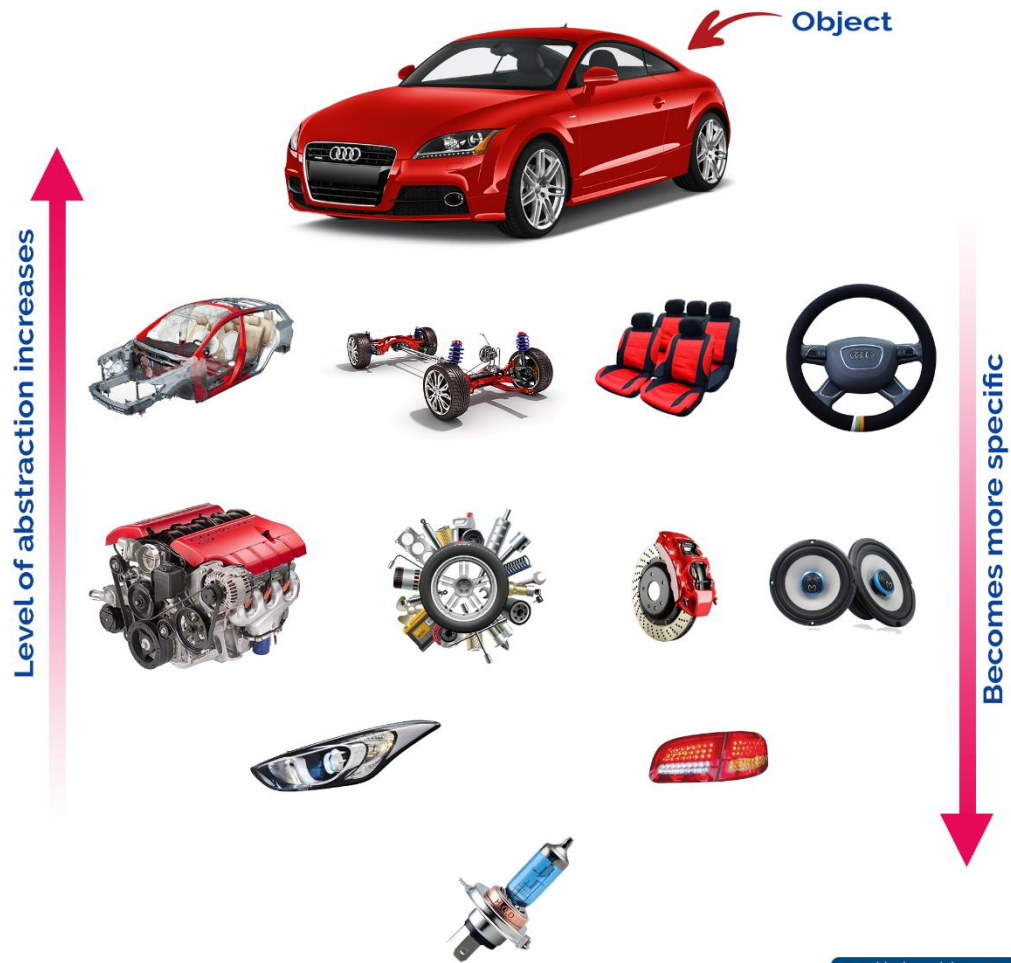
- Inheritance is the process of acquiring properties and behaviors from one object to another object or one class to another class. In inheritance, we derive a new class from the existing class. Here, the new class acquires the properties and behaviors from the existing class. In the inheritance concept, the class which provides properties is called as parent class and the class which receives the properties is called as child class. The parent class is also known as base class or super class. The child class is also known as derived class or sub class.
- In the inheritance, the properties and behaviors of base class are extended to its derived class, but the base class never receives properties or behaviors from its derived class.
- In Java programming language, the keyword `extends` is used to implement inheritance.
- **Polymorphism**



www.btechsmartclass.com

- Polymorphism is the process of defining the same method with different implementation. That means creating multiple methods with different behaviors.
- Java uses method overloading and method overriding to implement polymorphism.
- Method overloading - multiple methods with the same name but different parameters.
- Method overriding - multiple methods with the same name and same parameters.

- **Abstraction**



- Abstraction is hiding the internal details and showing only essential functionality. In the abstraction concept, we do not show the actual implementation to the end user, instead we provide only essential things. For example, if we want to drive a car, we do not need to know about the internal functionality like how wheel system works? how brake system works? how music system works? etc.

Benefits of Object Oriented Programming.

Benefits of OOP

- We can build the programs from standard working modules that communicate with one another, rather than having to start writing the code from scratch which leads to saving of development time and higher productivity,
- OOP language allows to break the program into the bit-sized problems that can be solved easily (one object at a time).
- The new technology promises greater programmer productivity, better quality of software and lesser maintenance cost.
- OOP systems can be easily upgraded from small to large systems.
- It is possible that multiple instances of objects co-exist without any interference,
- It is very easy to partition the work in a project based on objects.
- It is possible to map the objects in problem domain to those in the program.

- The principle of data hiding helps the programmer to build secure programs which cannot be invaded by the code in other parts of the program.
- By using inheritance, we can eliminate redundant code and extend the use of existing classes.
- Message passing techniques is used for communication between objects which makes the interface descriptions with external systems much simpler.
- The data-centered design approach enables us to capture more details of model in an implementable form.

Introduction to Java: Java buzzwords

Java is the most popular object-oriented programming language. Java has many advanced features, a list of key features is known as Java Buzz Words. The java team has listed the following terms as java buzz words.

- **Simple**
- **Secure**
- **Portable**
- **Object-oriented**
- **Robust**
- **Architecture-neutral (or) Platform Independent**
- **Multi-threaded**
- **Interpreted**
- **High performance**
- **Distributed**
- **Dynamic**

Simple

Java programming language is very simple and easy to learn, understand, and code. Most of the syntaxes in java follow basic programming language C and object-oriented programming concepts are similar to C++. In a java programming language, many complicated features like pointers, operator overloading, structures, unions, etc. have been removed. One of the most useful features is the garbage collector it makes java more simple.

Secure

Java is said to be more secure programming language because it does not have pointers concept, java provides a feature "applet" which can be embedded into a web application. The applet in java does not allow access to other parts of the computer, which keeps away from harmful programs like viruses and unauthorized access.

Portable

Portability is one of the core features of java which enables the java programs to run on any computer or operating system. For example, an applet developed using java runs on a wide variety of CPUs, operating systems, and browsers connected to the Internet.

Object-oriented

Java is said to be a pure object-oriented programming language. In java, everything is an object. It supports all the features of the object-oriented programming paradigm. The primitive data types java also implemented as objects using wrapper classes, but still, it allows primitive data types to archive high-performance.

Robust

Java is more robust because the java code can be executed on a variety of environments, java has a strong memory management mechanism (garbage collector), java is a strictly typed language, it has a strong set of exception handling mechanism, and many more.

Architecture-neutral (or) Platform Independent

Java has invented to archive "write once; run anywhere, any time, forever". The java provides JVM (Java Virtual Machine) to to archive architectural-neutral or platform-independent. The JVM allows the java program created using one operating system can be executed on any other operating system.

Multi-threaded

Java supports multi-threading programming, which allows us to write programs that do multiple operations simultaneously.

Interpreted

Java enables the creation of cross-platform programs by compiling into an intermediate representation called Java bytecode. The byte code is interpreted to any machine code so that it runs on the native machine.

High performance

Java provides high performance with the help of features like JVM, interpretation, and its simplicity.

Distributed

Java programming language supports TCP/IP protocols which enable the java to support the distributed environment of the Internet. Java also supports Remote Method Invocation (RMI), this feature enables a program to invoke methods across a network.

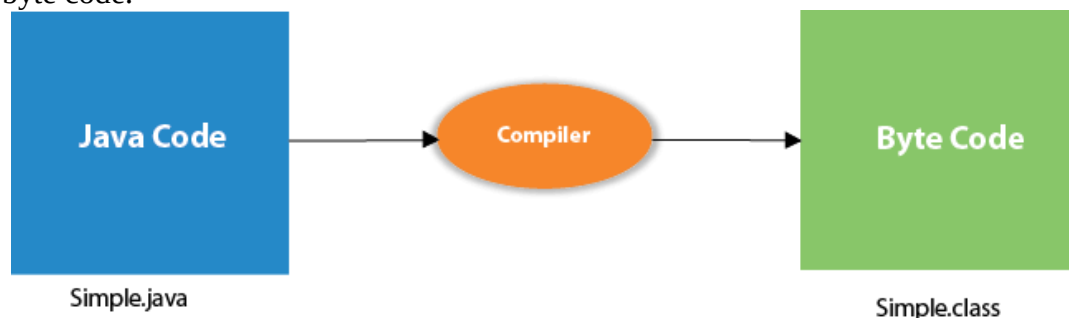
Dynamic

Java is said to be dynamic because the java byte code may be dynamically updated on a running system and it has a dynamic memory allocation and deallocation (objects and garbage collector).

Bytecode

Compilation Flow:

When we compile Java program using javac tool, the Java compiler converts the source code into byte code.



Java Programming Fundamentals: Applet and Application program using simple java program

Java Applet

Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

Drawback of Applet

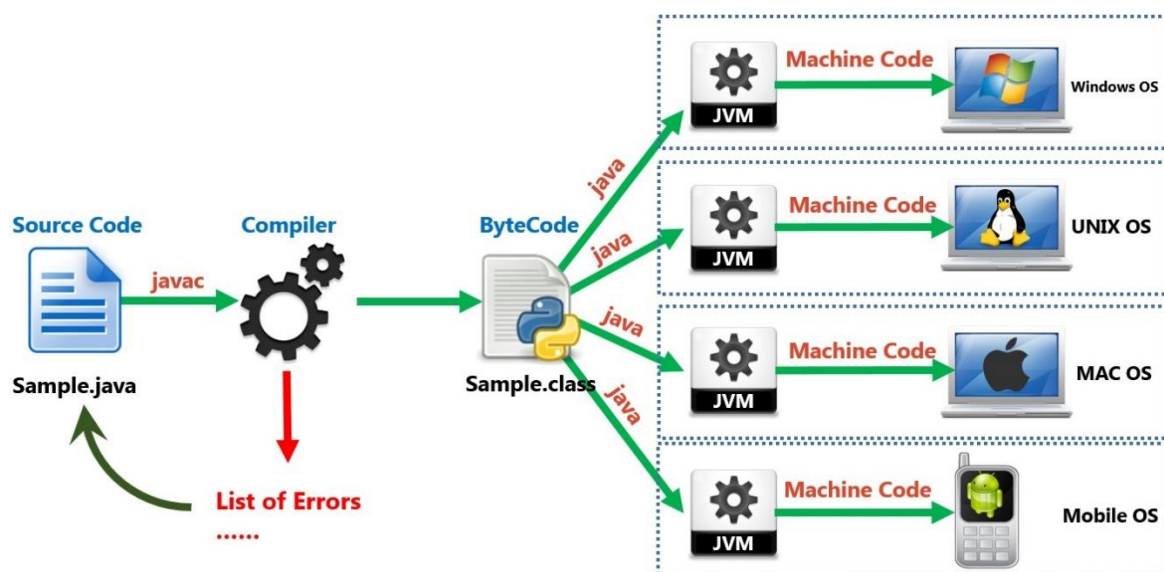
- Plugin is required at client browser to execute applet.
-

Application program using simple java program

Execution Process of Java Program

The following three steps are used to create and execute a java program.

- Create a source code (.java file).
- Compile the source code using javac command.
- Run or execute .class file using java command.



Creating Hello World Example

Let's create the hello java program:

1. **class** Simple{
2. **public static void** main(String args[]){
3. System.out.println("Hello Java");
4. }
5. }

Save the above file as Simple.java.

To compile:

javac Simple.java

To execute:

java Simple

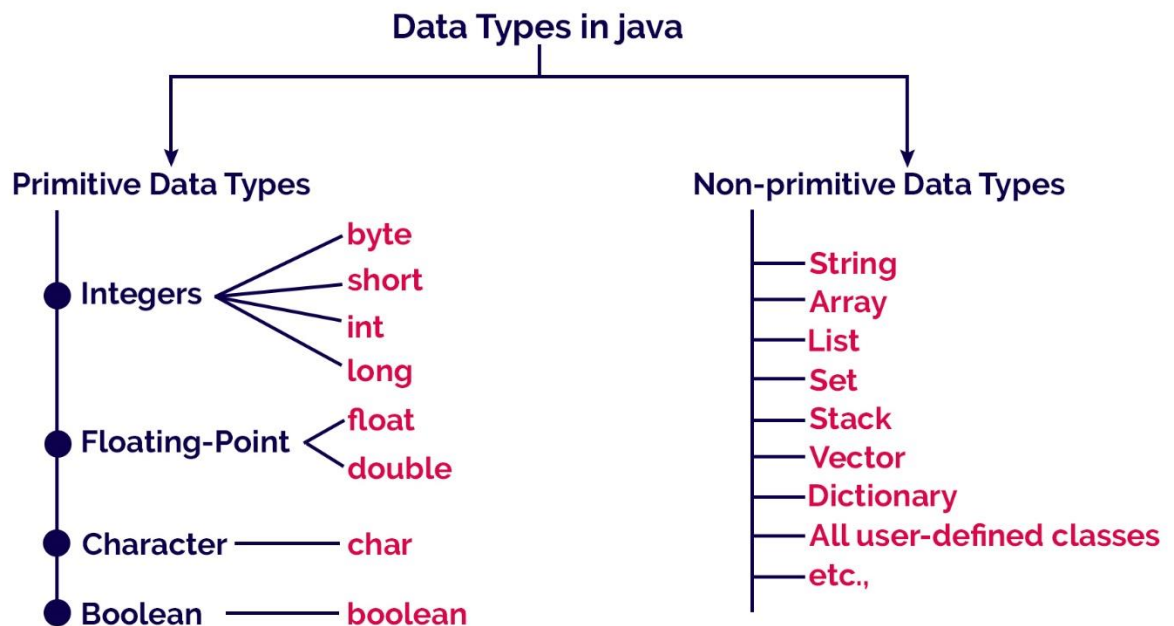
Output:

Hello Java

data types

Java programming language has a rich set of data types. The data type is a category of data stored in variables. In java, data types are classified into two types and they are as follows.

- **Primitive Data Types**
- **Non-primitive Data Types**



Primitive Data Types

The primitive data types are built-in data types and they specify the type of value stored in a variable and the memory size. The primitive data types do not have any additional methods. In java, primitive data types includes **byte**, **short**, **int**, **long**, **float**, **double**, **char**, and **boolean**. The following table provides more description of each primitive data type.

Data type	Meaning	Memory size	Range	Default Value
byte	Whole numbers	1 byte	-128 to +127	0
short	Whole numbers	2 bytes	-32768 to +32767	0
int	Whole numbers	4 bytes	-2,147,483,648 to +2,147,483,647	0
long	Whole numbers	8 bytes	-9,223,372,036,854,775,808 to +9,223,372,036,854,775,807	0L
float	Fractional numbers	4 bytes	-	0.0f
double	Fractional numbers	8 bytes	-	0.0d
char	Single character	2 bytes	0 to 65535	\u0000
boolean	unsigned char	1 bit	0 or 1	0 (false)

Let's look at the following example java program to illustrate primitive data types in java and their default values.

```

1 public class PrimitiveDataTypes {
2
3     byte i;
4     short j;
5     int k;
6     long l;
7     float m;
8     double n;
9     char ch;
10    boolean p;
11
12    public static void main(String[] args) {
13
14        PrimitiveDataTypes obj = new PrimitiveDataTypes();
15
16        System.out.println("i = " + obj.i + ", j = " + obj.j + ", k = " + obj.k + ", l = " + obj.l);
17        System.out.println("m = " + obj.m + ", n = " + obj.n);
18        System.out.println("ch = " + obj.ch);
19        System.out.println("p = " + obj.p);
20    }
21 }
22
23

```

Console Output:

```

i = 0, j = 0, k = 0, l = 0
m = 0.0, n = 0.0
ch = 
p = false

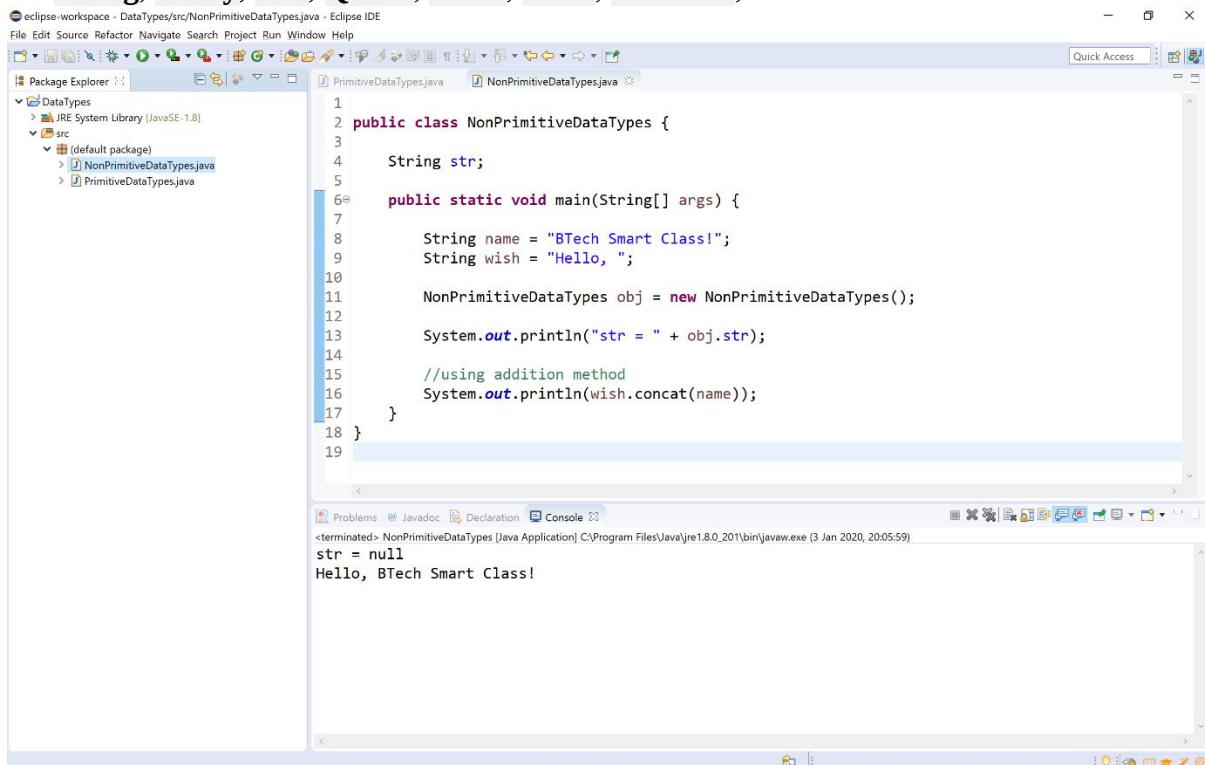
```

Non-primitive Data Types

In java, non-primitive data types are the reference data types or user-created data types. All non-primitive data types are implemented using object concepts. Every variable of the non-

primitive data type is an object. The non-primitive data types may use additional methods to perform certain operations. The default value of non-primitive data type variable is null.

In java, examples of non-primitive data types are **String, Array, List, Queue, Stack, Class, Interface**, etc.



The screenshot shows the Eclipse IDE with the following details:

- Package Explorer:** Shows the project structure with 'DataTypes' containing 'src' which has 'NonPrimitiveDataTypes.java' and 'PrimitiveDataTypes.java'.
- Editor:** Displays the code for 'NonPrimitiveDataTypes.java':

```
1
2 public class NonPrimitiveDataTypes {
3
4     String str;
5
6     public static void main(String[] args) {
7
8         String name = "BTech Smart Class!";
9         String wish = "Hello, ";
10
11         NonPrimitiveDataTypes obj = new NonPrimitiveDataTypes();
12
13         System.out.println("str = " + obj.str);
14
15         //using addition method
16         System.out.println(wish.concat(name));
17     }
18 }
19
```
- Console:** Shows the output of the program:

```
<terminated> NonPrimitiveDataTypes [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (3 Jan 2020, 20:05:59)
str = null
Hello, BTech Smart Class!
```

Primitive Vs Non-primitive Data Types

Primitive Data Type	Non-primitive Data Type
These are built-in data types	These are created by the users
Does not support additional methods	Support additional methods
Always has a value	It can be null
Starts with lower-case letter	Starts with upper-case letter
Size depends on the data type	Same size for all

Variables

A variable is a named memory location used to store a data value. A variable can be defined as a container that holds a data value.

In java, we use the following syntax to create variables.

Syntax

data_type variable_name;

(or)

data_type variable_name_1, variable_name_2,...;

(or)

data_type variable_name = value;

(or)

data_type variable_name_1 = value, variable_name_2 = value,...;

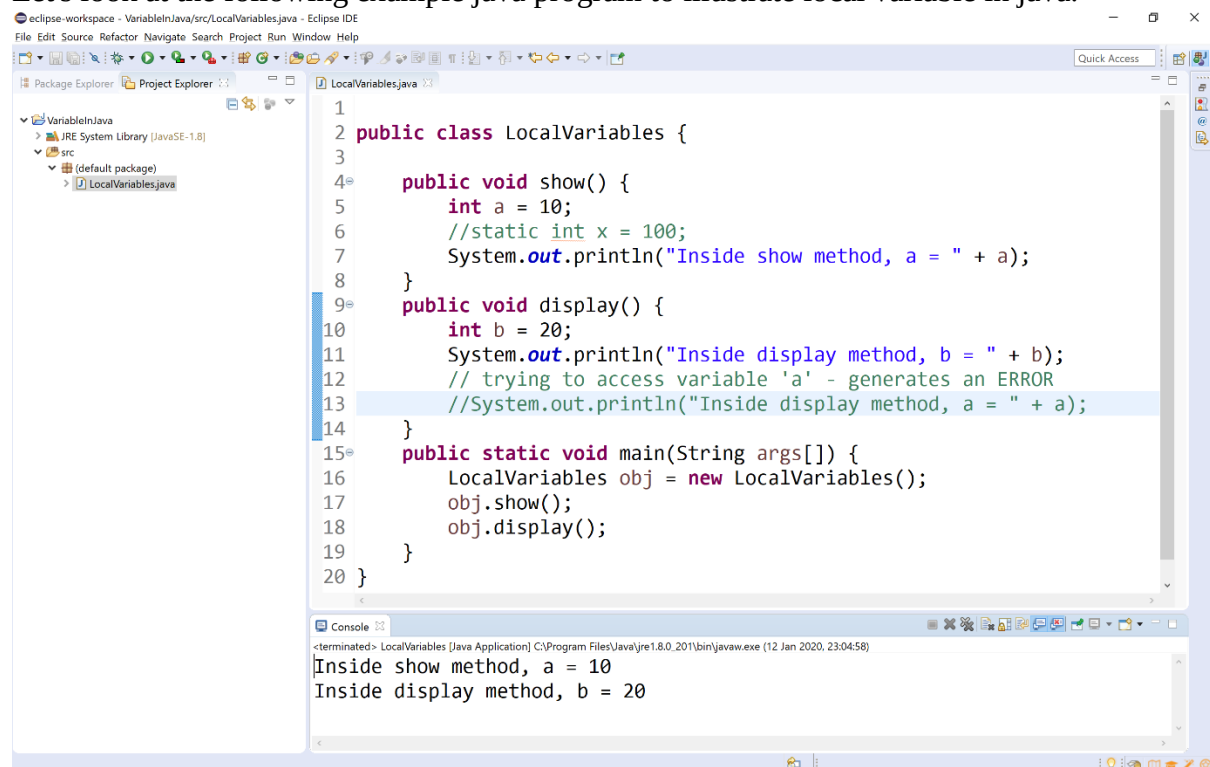
In java programming language variables are clasiffied as follows.

- **Local variables**
- **Instance variables or Member variables or Global variables**
- **Static variables or Class variables**
- **Final variables**

Local variables

The variables declared inside a method or a block are known as local variables. A local variable is visible within the method in which it is declared. The local variable is created when execution control enters into the method or block and destroyed after the method or block execution completed.

Let's look at the following example java program to illustrate local variable in java.



```
eclipse-workspace - VariableInJava/src/LocalVariables.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer | Project Explorer | LocalVariables.java
VariableInJava
├── JRE System Library [JavaSE-1.8]
├── src
│   └── (default package)
│       └── LocalVariables.java

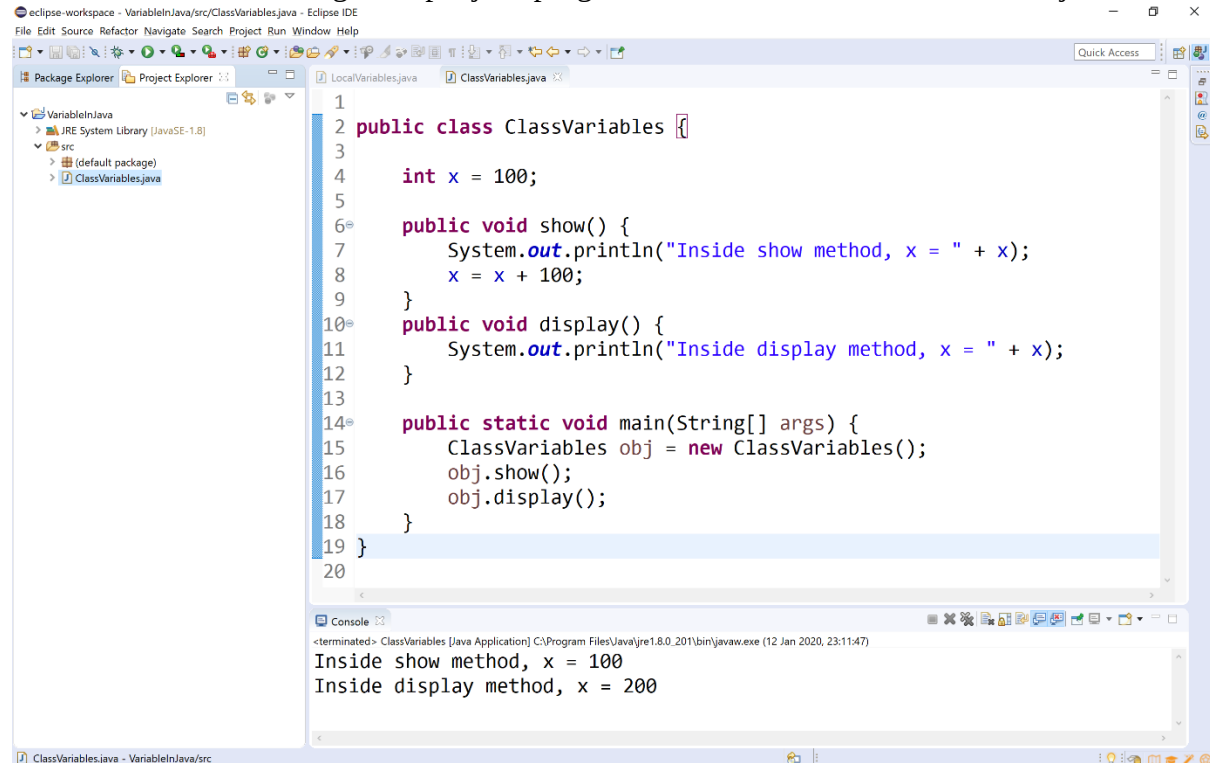
1
2 public class LocalVariables {
3
4     public void show() {
5         int a = 10;
6         //static int x = 100;
7         System.out.println("Inside show method, a = " + a);
8     }
9     public void display() {
10        int b = 20;
11        System.out.println("Inside display method, b = " + b);
12        // trying to access variable 'a' - generates an ERROR
13        //System.out.println("Inside display method, a = " + a);
14    }
15    public static void main(String args[]) {
16        LocalVariables obj = new LocalVariables();
17        obj.show();
18        obj.display();
19    }
20 }
```

```
<terminated> LocalVariables [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (12 Jan 2020, 23:04:58)
Inside show method, a = 10
Inside display method, b = 20
```

Instance variables or member variables or global variables

The variables declared inside a class and outside any method, constructor or block are known as instance variables or member variables. These variables are visible to all the methods of the class. The changes made to these variables by method affects all the methods in the class. These variables are created separate copy for every object of that class.

Let's look at the following example java program to illustrate instance variable in java.



The screenshot shows the Eclipse IDE with a project named 'VariableInJava'. The 'src' folder contains a file named 'ClassVariables.java'. The code in this file is as follows:

```
1
2 public class ClassVariables {
3
4     int x = 100;
5
6     public void show() {
7         System.out.println("Inside show method, x = " + x);
8         x = x + 100;
9     }
10    public void display() {
11        System.out.println("Inside display method, x = " + x);
12    }
13
14    public static void main(String[] args) {
15        ClassVariables obj = new ClassVariables();
16        obj.show();
17        obj.display();
18    }
19 }
20
```

The console output shows the execution of the program:

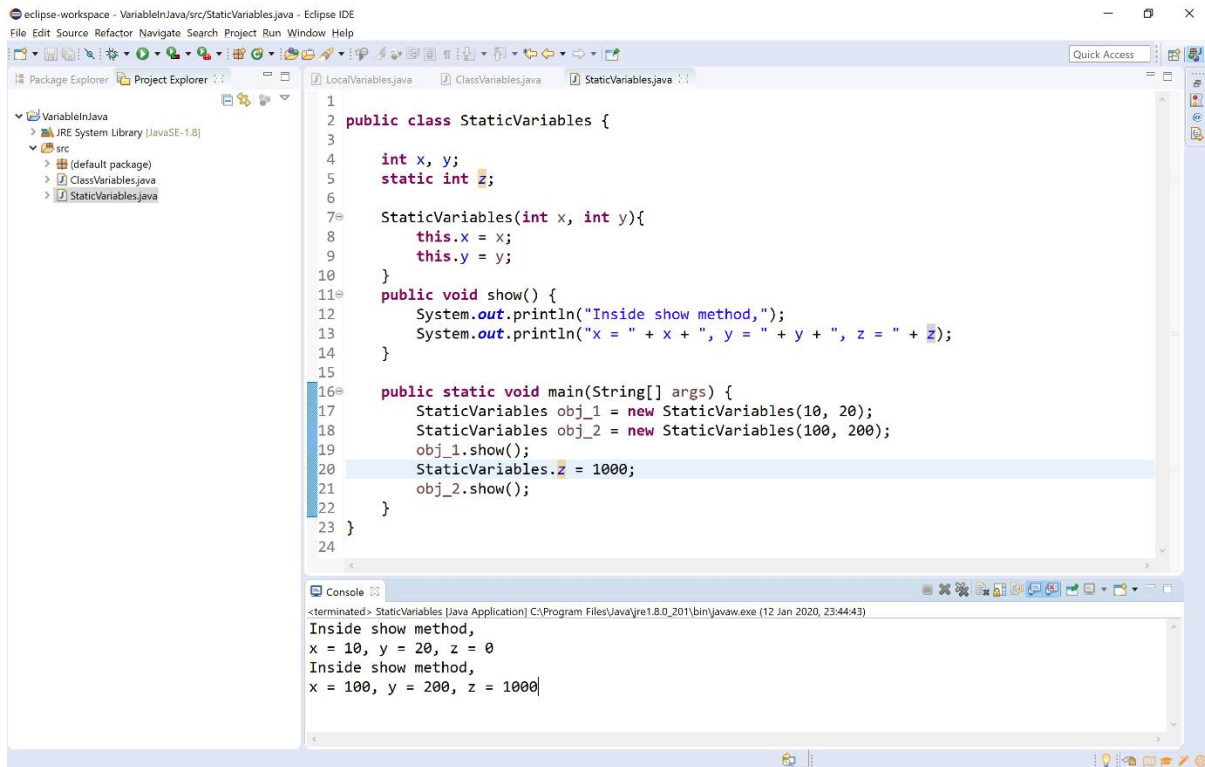
```
<terminated> ClassVariables [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (12 Jan 2020, 23:11:47)
Inside show method, x = 100
Inside display method, x = 200
```

Static variables or Class variables

A static variable is a variable that declared using **static** keyword. The instance variables can be static variables but local variables can not. Static variables are initialized only once, at the start of the program execution. The static variable only has one copy per class irrespective of how many objects we create.

The static variable is access by using class name.

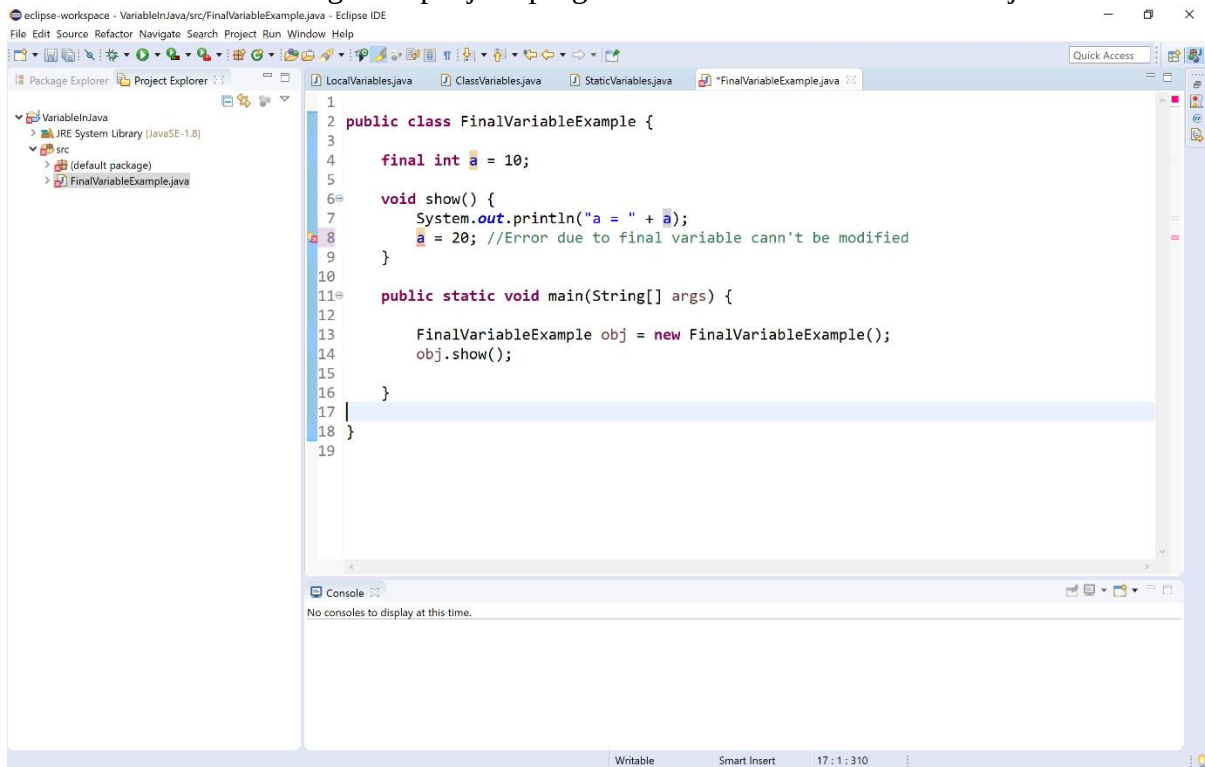
Let's look at the following example java program to illustrate static variable in java.



Final variables

A final variable is a variable that declared using **final** keyword. The final variable is initialized only once, and does not allow any method to change it's value again. The variable created using **final** keyword acts as constant. All variables like local, instance, and static variables can be final variables.

Let's look at the following example java program to illustrate final variable in java.



Arrays

An array is a collection of similar data values with a single name. An array can also be defined as, a special type of variable that holds multiple values of the same data type at a time.

In java, arrays are objects and they are created dynamically using **new** operator. Every array in java is organized using index values. The index value of an array starts with '0' and ends with 'size-1'. We use the index value to access individual elements of an array.

In java, there are two types of arrays and they are as follows.

- **One Dimensional Array**
- **Multi Dimensional Array**

Creating an array

In the java programming language, an array must be created using new operator and with a specific size. The size must be an integer value but not a byte, short, or long. We use the following syntax to create an array.

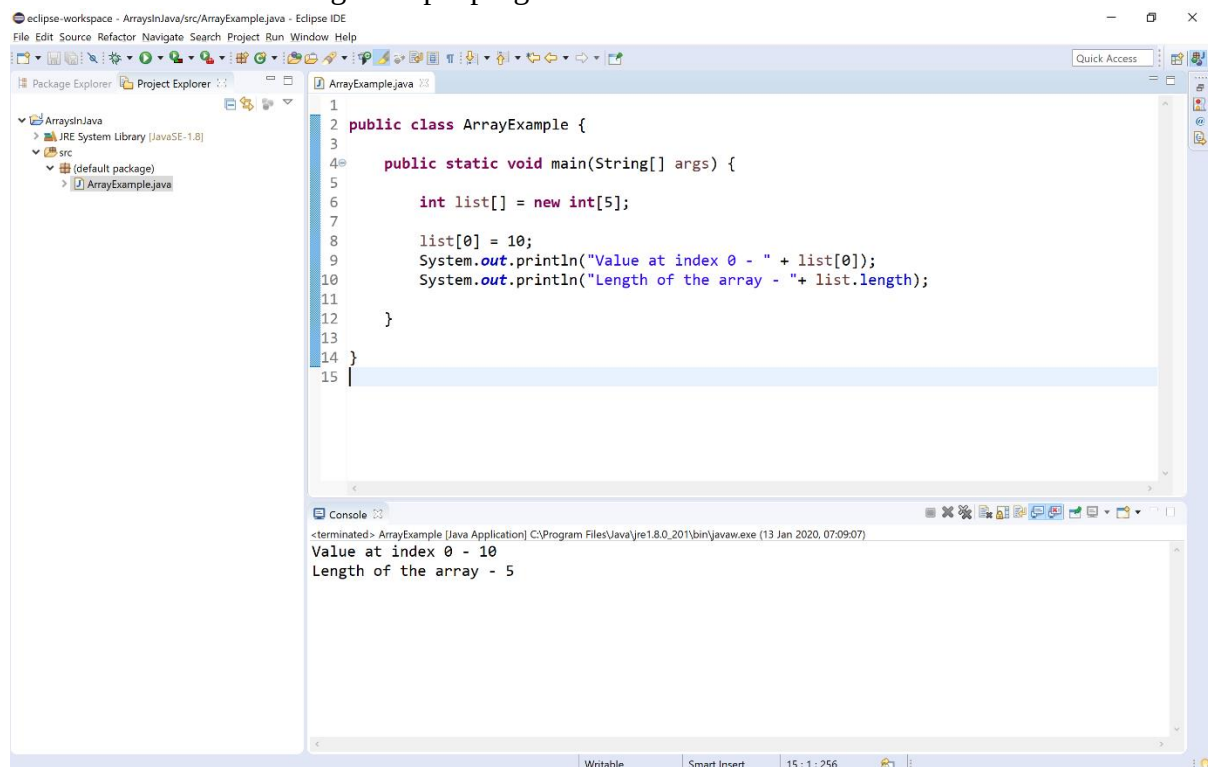
Syntax

```
data_type array_name[ ] = new data_type[size];
```

(or)

```
data_type[ ] array_name = new data_type[size];
```

Let's look at the following example program.



The screenshot shows the Eclipse IDE with a project named 'ArraysInJava'. The 'src' folder contains a file named 'ArrayExample.java'. The code in the file is as follows:

```
1 public class ArrayExample {
2
3
4     public static void main(String[] args) {
5
6         int list[] = new int[5];
7
8         list[0] = 10;
9         System.out.println("Value at index 0 - " + list[0]);
10        System.out.println("Length of the array - " + list.length);
11
12    }
13
14 }
15
```

The console output at the bottom shows the execution results:

```
<terminated> ArrayExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (13 Jan 2020, 07:09:07)
Value at index 0 - 10
Length of the array - 5
```

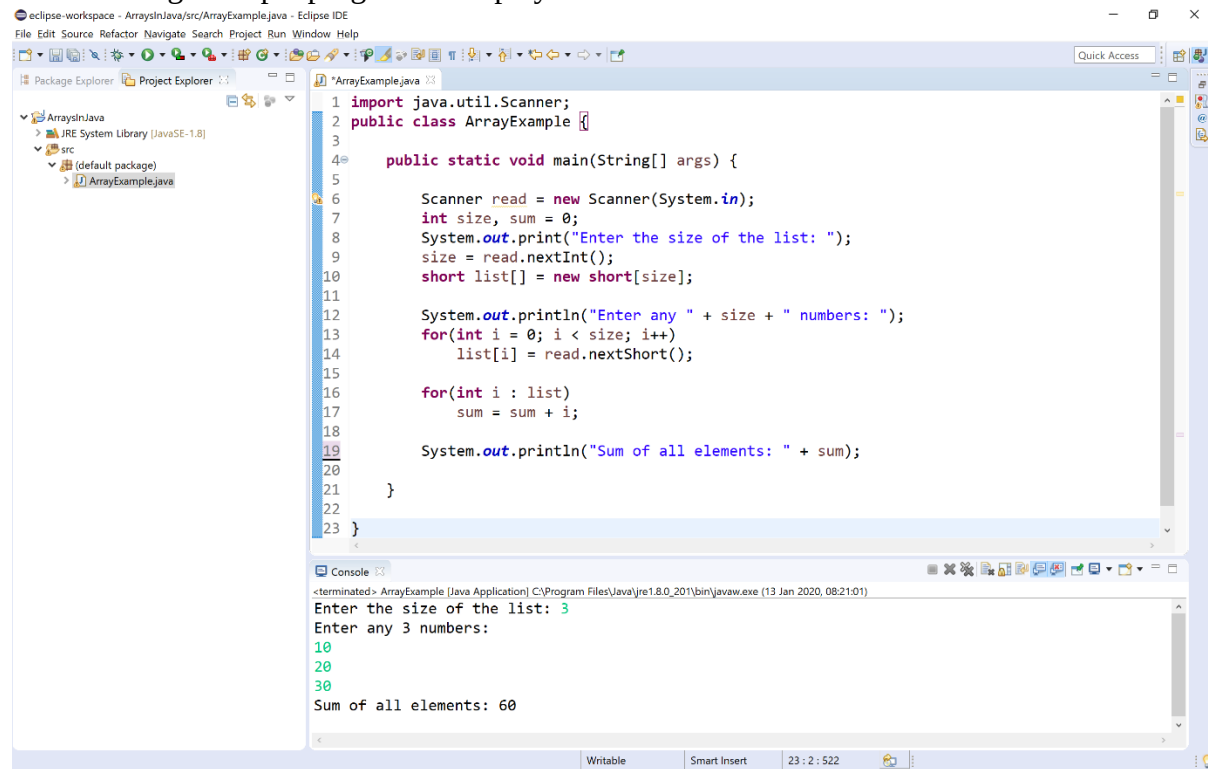
In java, an array can also be initialized at the time of its declaration. When an array is initialized at the time of its declaration, it need not specify the size of the array and use of the new operator. Here, the size is automatically decided based on the number of values that are initialized.

Example

```
int list[ ] = {10, 20, 30, 40, 50};
```

Looping through an array

An entire array is accessed using either simple **for statement** or **for-each statement**. Look at the following example program to display sum of all the elements in a list.



```
1 import java.util.Scanner;
2 public class ArrayExample {
3
4     public static void main(String[] args) {
5
6         Scanner read = new Scanner(System.in);
7         int size, sum = 0;
8         System.out.print("Enter the size of the list: ");
9         size = read.nextInt();
10        short list[] = new short[size];
11
12        System.out.println("Enter any " + size + " numbers: ");
13        for(int i = 0; i < size; i++)
14            list[i] = read.nextShort();
15
16        for(int i : list)
17            sum = sum + i;
18
19        System.out.println("Sum of all elements: " + sum);
20    }
21 }
22
23 }
```

Console Output:

```
<terminated> ArrayExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (13 Jan 2020, 08:21:01)
Enter the size of the list: 3
Enter any 3 numbers:
10
20
30
Sum of all elements: 60
```

Multidimensional Array

In java, we can create an array with multiple dimensions. We can create 2-dimensional, 3-dimensional, or any dimensional array.

In Java, multidimensional arrays are arrays of arrays. To create a multidimensional array variable, specify each additional index using another set of square brackets. We use the following syntax to create two-dimensional array.

Syntax

```
data_type array_name[ ][ ] = new data_type[rows][columns];
```

(or)

```
data_type[ ][ ] array_name = new data_type[rows][columns];
```

When we create a two-dimensional array, it is created with a separate index for rows and columns. The individual element is accessed using the respective row index followed by the column index. A multidimensional array can be initialized while it has been created using the following syntax.

Syntax

```
data_type array_name[ ][ ] = {{value1, value2}, {value3, value4}, {value5, value6},...};
```

When an array is initialized at the time of declaration, it need not specify the size of the array and use of the new operator. Here, the size is automatically decided based on the number of values that are initialized.

Example

```
int matrix_a[ ][ ] = {{1, 2},{3, 4},{5, 6}};
```

The above statement creates a two-dimensional array of three rows and two columns.

Operators

An operator is a symbol used to perform arithmetic and logical operations. Java provides a rich set of operators.

In java, operators are clasified into the following four types.

- **Arithmetic Operqators**
- **Relational (or) Comparision Operators**
- **Logical Operators**
- **Assignment Operators**
- **Bitwise Operators**
- **Conditional Operators**

Let's look at each operator in detail.

Arithmetic Operators

In java, arithmetic operators are used to performing basic mathematical operations like addition, subtraction, multiplication, division, modulus, increment, decrement, etc.,

Operator	Meaning	Example
+	Addition	10 + 5 = 15
-	Subtraction	10 - 5 = 5
*	Multiplication	10 * 5 = 50
/	Division	10 / 5 = 2
%	Modulus - Remainder of the Division	5 % 2 = 1
++	Increment	a++
--	Decrement	a--

Let's look at the following example program.

```

1 public class ArithmeticOperators {
2
3     public static void main(String[] args) {
4
5         int a = 10, b = 20, result;
6
7         System.out.println("a = " + a + ", b = " + b);
8
9
10        result = a + b;
11        System.out.println("Addition : " + a + " + " + b + " = " + result);
12
13        result = a - b;
14        System.out.println("Subtraction : " + a + " - " + b + " = " + result);
15
16        result = a * b;
17        System.out.println("Multiplucation : " + a + " * " + b + " = " + result);
18
19        result = b / a;
20        System.out.println("Division : " + b + " / " + a + " = " + result);
21
22        result = b % a;
23        System.out.println("Modulus : " + b + " % " + a + " = " + result);
24
25        result = ++a;
26        System.out.println("Pre-increment : ++a = " + result);
27
28        result = b--;
29        System.out.println("Post-decrement : b-- = " + result);
30    }
31 }

```

```

<terminated> ArithmeticOperators [Java Application] C:\Program Files\Ja
a = 10, b = 20
Addition : 10 + 20 = 30
Subtraction : 10 - 20 = -10
Multiplucation : 10 * 20 = 200
Division : 20 / 10 = 2
Modulus : 20 % 10 = 0
Pre-increment : ++a = 11
Post-decrement : b-- = 20

```

Relational Operators (<, >, <=, >=, ==, !=)

The relational operators are the symbols that are used to compare two values. That means the relational operators are used to check the relationship between two values. Every relational operator has two possible results either **TRUE** or **FALSE**. In simple words, the relational operators are used to define conditions in a program. The following table provides information about relational operators.

Operator	Meaning	Example
<	Returns TRUE if the first value is smaller than second value otherwise returns FALSE	10 < 5 is FALSE
>	Returns TRUE if the first value is larger than second value otherwise returns FALSE	10 > 5 is TRUE
<=	Returns TRUE if the first value is smaller than or equal to second value otherwise returns FALSE	10 <= 5 is FALSE
>=	Returns TRUE if the first value is larger than or equal to second value otherwise returns FALSE	10 >= 5 is TRUE
==	Returns TRUE if both values are equal otherwise returns FALSE	10 == 5 is FALSE
!=	Returns TRUE if both values are not equal otherwise returns FALSE	10 != 5 is TRUE

Look at the following example program.

The screenshot shows the Eclipse IDE with a Java file named `RelationalOperators.java`. The code defines a public class `RelationalOperators` with a `main` method. Inside `main`, a `boolean` variable `a` is declared and assigned the result of various relational expressions. Each expression is printed to the console using `System.out.println`.

```
1 public class RelationalOperators {  
2  
3     public static void main(String[] args) {  
4  
5         boolean a;  
6  
7         a = 10<5;  
8         System.out.println("10 < 5 is " + a);  
9  
10        a = 10>5;  
11        System.out.println("10 > 5 is " + a);  
12  
13        a = 10<=5;  
14        System.out.println("10 <= 5 is " + a);  
15  
16        a = 10>=5;  
17        System.out.println("10 >= 5 is " + a);  
18  
19        a = 10==5;  
20        System.out.println("10 == 5 is " + a);  
21  
22        a = 10!=5;  
23        System.out.println("10 != 5 is " + a);  
24    }  
25 }  
26  
27 }  
28
```

The console output on the right shows the results of these expressions:

```
<terminated> RelationalOperators [Java Application] C:\Program Files\Java  
10 < 5 is false  
10 > 5 is true  
10 <= 5 is false  
10 >= 5 is true  
10 == 5 is false  
10 != 5 is true
```

Logical Operators

The logical operators are the symbols that are used to combine multiple conditions into one condition. The following table provides information about logical operators.

Operator	Meaning	Example
&	Logical AND - Returns TRUE if all conditions are TRUE otherwise returns FALSE	false & true => false
	Logical OR - Returns FALSE if all conditions are FALSE otherwise returns TRUE	false true => true
^	Logical XOR - Returns FALSE if all conditions are same otherwise returns TRUE	true ^ true => false
!	Logical NOT - Returns TRUE if condition is FALSE and returns FALSE if it is TRUE	!false => true
&&	short-circuit AND - Similar to Logical AND (&), but once a decision is finalized it does not evaluate remaining.	false & true => false
	short-circuit OR - Similar to Logical OR (), but once a decision is finalized it does not evaluate remaining.	false true => true

Look at the following example program.

The screenshot shows the Eclipse IDE with a Java file named `LogicalOperators.java`. The code defines a `LogicalOperators` class with a `main` method. Inside `main`, variables `x`, `y`, and `z` are initialized to 10, 20, and 0 respectively. A `boolean` variable `a` is set to `true`. Then, a logical AND operation is performed: `a = x > y && (z = x + y) > 15;`. This operation updates `a` to `false` and `z` to 30. The console output shows the state of `a` and `z` before and after the operation.

```

1 public class LogicalOperators {
2
3
4     public static void main(String[] args) {
5
6         int x = 10, y = 20, z = 0;
7         boolean a = true;
8
9         a = x > y && (z = x + y) > 15;
10        System.out.println("a = " + a + ", and z = " + z);
11
12        a = x > y & (z = x + y) > 15;
13        System.out.println("a = " + a + ", and z = " + z);
14    }
15
16
17 }

```

Console Output:

```

<terminated> LogicalOperators [Java Application] C:\Program Files\Java\jre1.8.0
a = false, and z = 0
a = false, and z = 30

```

Assignment Operators

The assignment operators are used to assign right-hand side value (Rvalue) to the left-hand side variable (Lvalue). The assignment operator is used in different variants along with arithmetic operators. The following table describes all the assignment operators in the java programming language.

Operator	Meaning	Example
=	Assign the right-hand side value to left-hand side variable	A = 15
+=	Add both left and right-hand side values and store the result into left-hand side variable	A += 10
-=	Subtract right-hand side value from left-hand side variable value and store the result into left-hand side variable	A -= B
*=	Multiply right-hand side value with left-hand side variable value and store the result into left-hand side variable	A *= B
/=	Divide left-hand side variable value with right-hand side variable value and store the result into the left-hand side variable	A /= B
%=	Divide left-hand side variable value with right-hand side variable value and store the remainder into the left-hand side variable	A %= B
&=	Logical AND assignment	-
=	Logical OR assignment	-
^=	Logical XOR assignment	-

Look at the following example program.

```

1 public class AssignmentOperators {
2
3
4     public static void main(String[] args) {
5
6         int a = 10, b = 20, c;
7         boolean x = true;
8
9         System.out.println("a = " + a + ", b = " + b);
10
11         a += b;
12         System.out.println("a = " + a);
13
14         a -= b;
15         System.out.println("a = " + a);
16
17         a *= b;
18         System.out.println("a = " + a);
19
20         a /= b;
21         System.out.println("a = " + a);
22
23         a %= b;
24         System.out.println("a = " + a);
25
26         x |= (a > b);
27         System.out.println("x = " + x);
28
29         x &= (a > b);
30         System.out.println("x = " + x);
31     }
32 }

```

```

<terminated> AssignmentOperators [Java Application] C:\Program Files\Java\re
a = 10, b = 20
a = 30
a = 10
a = 200
a = 10
a = 10
x = true
x = false

```

Bitwise Operators

The bitwise operators are used to perform bit-level operations in the java programming language. When we use the bitwise operators, the operations are performed based on binary values. The following table describes all the bitwise operators in the java programming language.

Let us consider two variables A and B as A = 25 (11001) and B = 20 (10100).

Operator	Meaning	Example
&	the result of Bitwise AND is 1 if all the bits are 1 otherwise it is 0	A & B ⇒ 16 (10000)
	the result of Bitwise OR is 0 if all the bits are 0 otherwise it is 1	A B ⇒ 29 (11101)
^	the result of Bitwise XOR is 0 if all the bits are same otherwise it is 1	A ^ B ⇒ 13 (01101)
~	the result of Bitwise once complement is negation of the bit (Flipping)	~A ⇒ 6 (00110)
<<	the Bitwise left shift operator shifts all the bits to the left by the specified number of positions	A << 2 ⇒ 100 (1100100)
>>	the Bitwise right shift operator shifts all the bits to the right by the specified number of positions	A >> 2 ⇒ 6 (00110)

Look at the following example program.

The screenshot shows the Eclipse IDE with a Java file named `BitwiseOperators.java`. The code defines a `public class BitwiseOperators` with a `main` method. Inside the `main` method, two integer variables `a` and `b` are initialized to 25 and 20 respectively. The program then prints the results of several bitwise operations: `a & b`, `a | b`, `a ^ b`, `~a`, `a >> 2`, `a << 2`, and `a >>> 2`. The console output on the right shows the following results:

```
<terminated> BitwiseOperators [Java Application] C:\Program Files\Java\jre1.8.0_65
25 & 20 = 16
25 | 20 = 29
25 ^ 20 = 13
~25 = -26
25 >> 2 = 6
25 << 2 = 100
25 >>> 2 = 6
```

Conditional Operators

The conditional operator is also called a **ternary operator** because it requires three operands. This operator is used for decision making. In this operator, first, we verify a condition, then we perform one operation out of the two operations based on the condition result. If the condition is TRUE the first option is performed, if the condition is FALSE the second option is performed. The conditional operator is used with the following syntax.

Syntax

```
Condition ? TRUE Part : FALSE Part;
```

Look at the following example program.

The screenshot shows the Eclipse IDE with a Java file named `ConditionalOperator.java`. The code defines a class `ConditionalOperator` with a `main` method. Inside `main`, variables `a` and `b` are initialized to 10 and 20 respectively. A conditional expression `c = (a > b) ? a : b;` is used to assign the value of `a` to `c` because `a` is greater than `b`. Finally, `System.out.println("c = " + c);` prints the result. The console on the right shows the output: `c = 20`.

```
1 public class ConditionalOperator {
2
3
4     public static void main(String[] args) {
5
6         int a = 10, b = 20, c;
7
8         c = (a > b) ? a : b;
9
10        System.out.println("c = " + c);
11    }
12
13 }
14
```

Console: `c = 20`

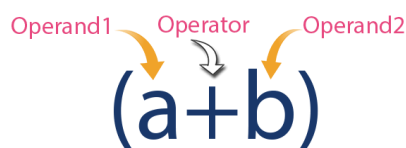
Expressions

In any programming language, if we want to perform any calculation or to frame any condition etc., we use a set of symbols to perform the task. These set of symbols makes an expression. In the java programming language, an expression is defined as follows.

An expression is a collection of operators and operands that represents a specific value.

In the above definition, an **operator** is a symbol that performs tasks like arithmetic operations, logical operations, and conditional operations, etc. **Operands** are the values on which the operators perform the task. Here operand can be a direct value or variable or address of memory location.

Example



control statements

In java, the default execution flow of a program is a sequential order. But the sequential order of execution flow may not be suitable for all situations. Sometimes, we may want to jump from line to another line, we may want to skip a part of the program, or sometimes we may want to execute a part of the program again and again. To solve this problem, java provides control statements.

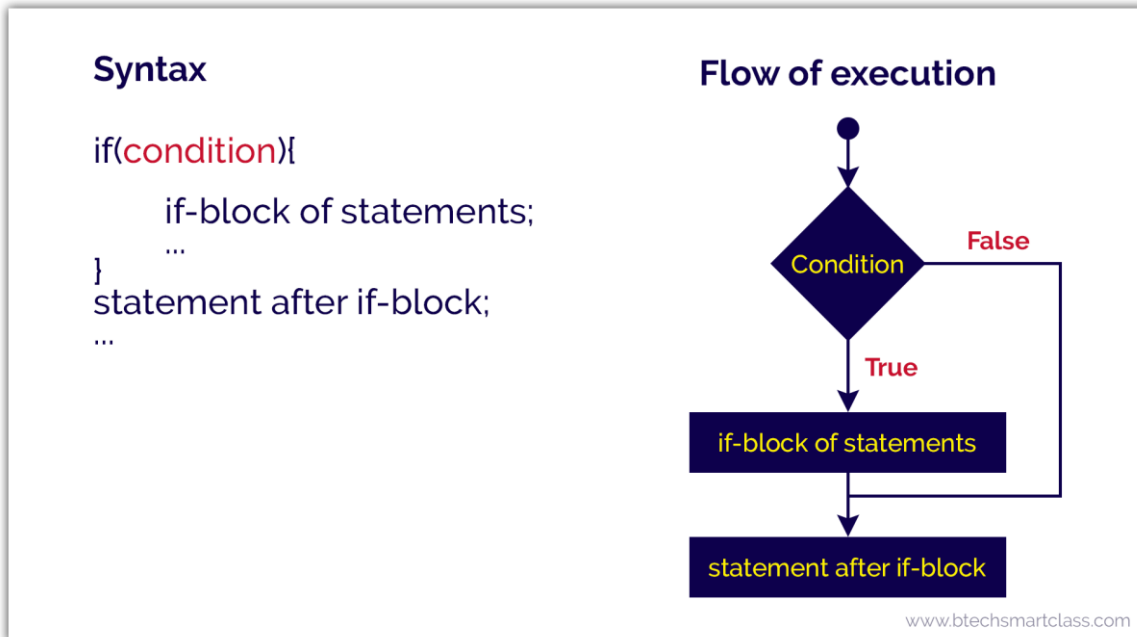
Java provides the following selection statements.

- if statement
- if-else statement
- nested if statement
- if-else if statement

- switch statement

if statement in java

In java, we use the if statement to test a condition and decide the execution of a block of statements based on that condition result. The if statement checks, the given condition then decides the execution of a block of statements. If the condition is True, then the block of statements is executed and if it is False, then the block of statements is ignored. The syntax and execution flow of if the statement is as follows.



Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java file named `IfStatementTest.java`. The code is as follows:

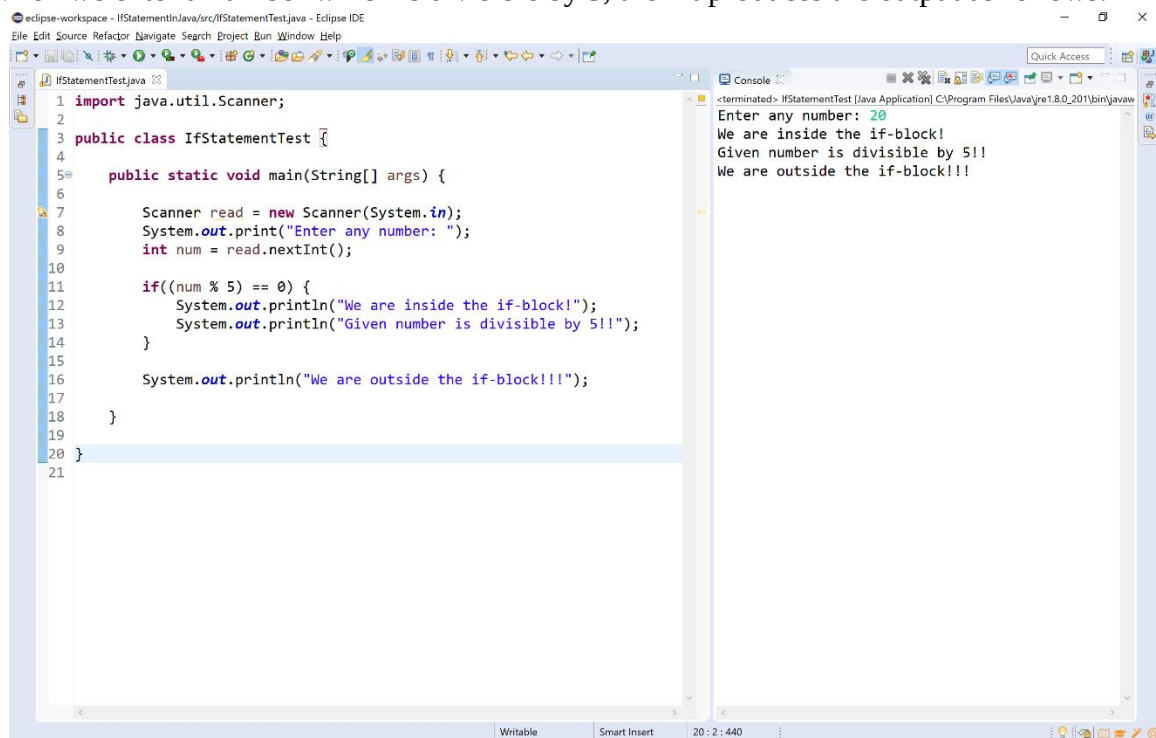
```
1 import java.util.Scanner;  
2  
3 public class IfStatementTest {  
4  
5     public static void main(String[] args) {  
6  
7         Scanner read = new Scanner(System.in);  
8         System.out.print("Enter any number: ");  
9         int num = read.nextInt();  
10  
11         if((num % 5) == 0) {  
12             System.out.println("We are inside the if-block!");  
13             System.out.println("Given number is divisible by 5!!!");  
14         }  
15  
16         System.out.println("We are outside the if-block!!!");  
17  
18     }  
19  
20 }  
21
```

The console output shows the program execution with the input "12":

```
<terminated> IfStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw  
Enter any number: 12  
We are outside the if-block!!!
```

In the above execution, the number 12 is not divisible by 5. So, the condition becomes False and the condition is evaluated to False. Then the if statement ignores the execution of its block of statements.

When we enter a number which is divisible by 5, then it produces the output as follows.



The screenshot shows the Eclipse IDE with a Java file named `IfStatementTest.java`. The code is as follows:

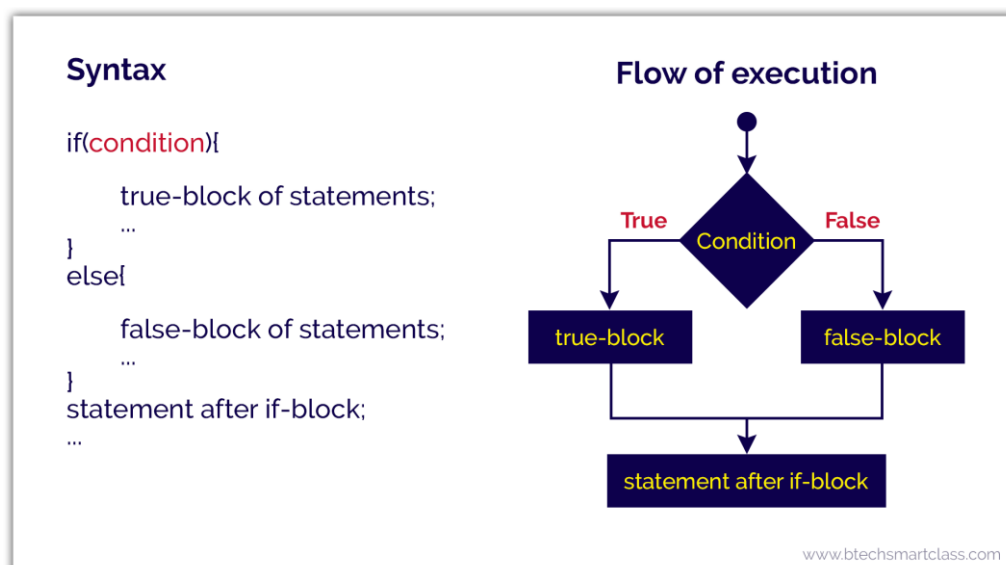
```
1 import java.util.Scanner;
2
3 public class IfStatementTest {
4
5     public static void main(String[] args) {
6
7         Scanner read = new Scanner(System.in);
8         System.out.print("Enter any number: ");
9         int num = read.nextInt();
10
11         if((num % 5) == 0) {
12             System.out.println("We are inside the if-block!");
13             System.out.println("Given number is divisible by 5!!");
14         }
15
16         System.out.println("We are outside the if-block!!!");
17     }
18 }
19
20
21
```

The console output shows the following:

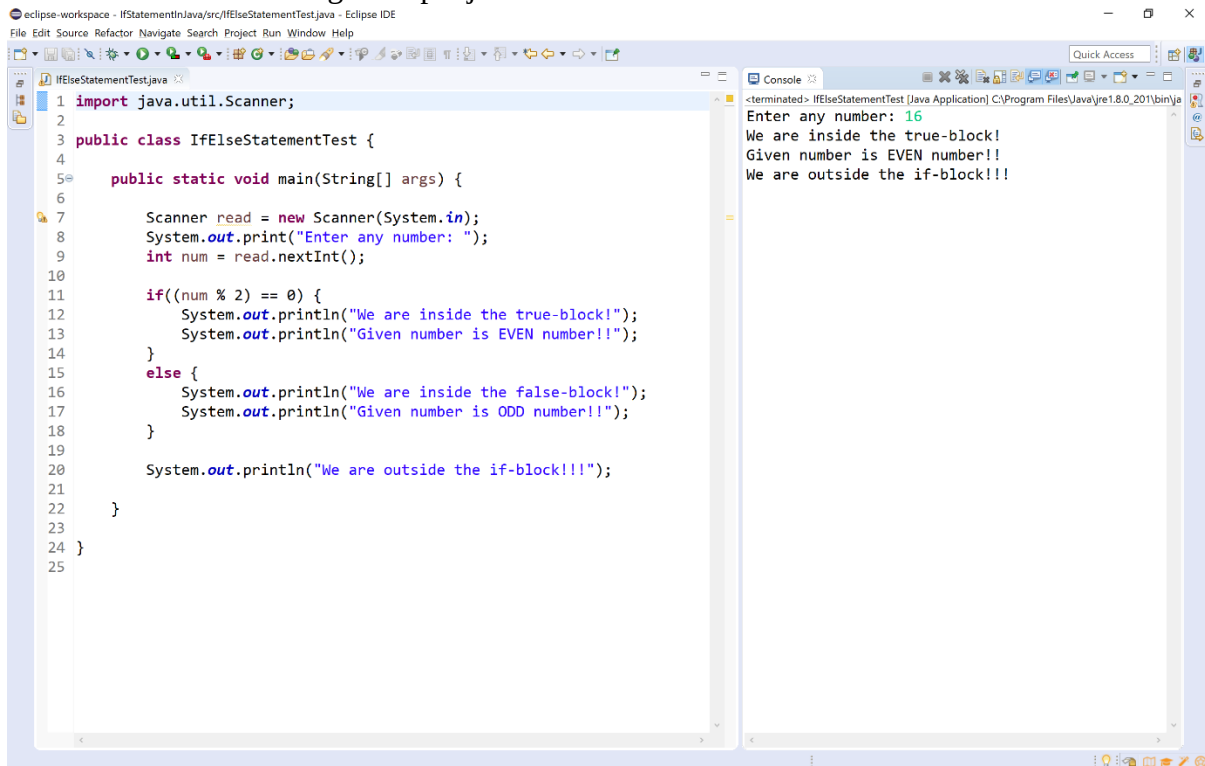
```
<terminated> IfStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw
Enter any number: 20
We are inside the if-block!
Given number is divisible by 5!!
We are outside the if-block!!!
```

if-else statement in java

In java, we use the if-else statement to test a condition and pick the execution of a block of statements out of two blocks based on that condition result. The if-else statement checks the given condition then decides which block of statements to be executed based on the condition result. If the condition is True, then the true block of statements is executed and if it is False, then the false block of statements is executed. The syntax and execution flow of if-else statement is as follows.



Let's look at the following example java code.



The screenshot shows the Eclipse IDE with a Java project named 'IfElseStatementTest'. The main editor displays the source code for 'IfElseStatementTest.java'. The code imports 'java.util.Scanner', defines a public class 'IfElseStatementTest', and contains a 'main' method. Inside the 'main' method, a 'Scanner' object is created to read input from 'System.in'. The user has entered '16'. The code checks if the number is even using 'num % 2 == 0'. Since 16 is even, it prints 'We are inside the true-block!' and 'Given number is EVEN number!!'. Finally, it prints 'We are outside the if-block!!!'.

```
1 import java.util.Scanner;
2
3 public class IfElseStatementTest {
4
5     public static void main(String[] args) {
6
7         Scanner read = new Scanner(System.in);
8         System.out.print("Enter any number: ");
9         int num = read.nextInt();
10
11         if((num % 2) == 0) {
12             System.out.println("We are inside the true-block!");
13             System.out.println("Given number is EVEN number!!");
14         }
15         else {
16             System.out.println("We are inside the false-block!");
17             System.out.println("Given number is ODD number!!");
18         }
19
20         System.out.println("We are outside the if-block!!!");
21     }
22 }
23
24 }
25
```

The console window on the right shows the execution output:

```
<terminated> IfElseStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\java
Enter any number: 16
We are inside the true-block!
Given number is EVEN number!!
We are outside the if-block!!!
```

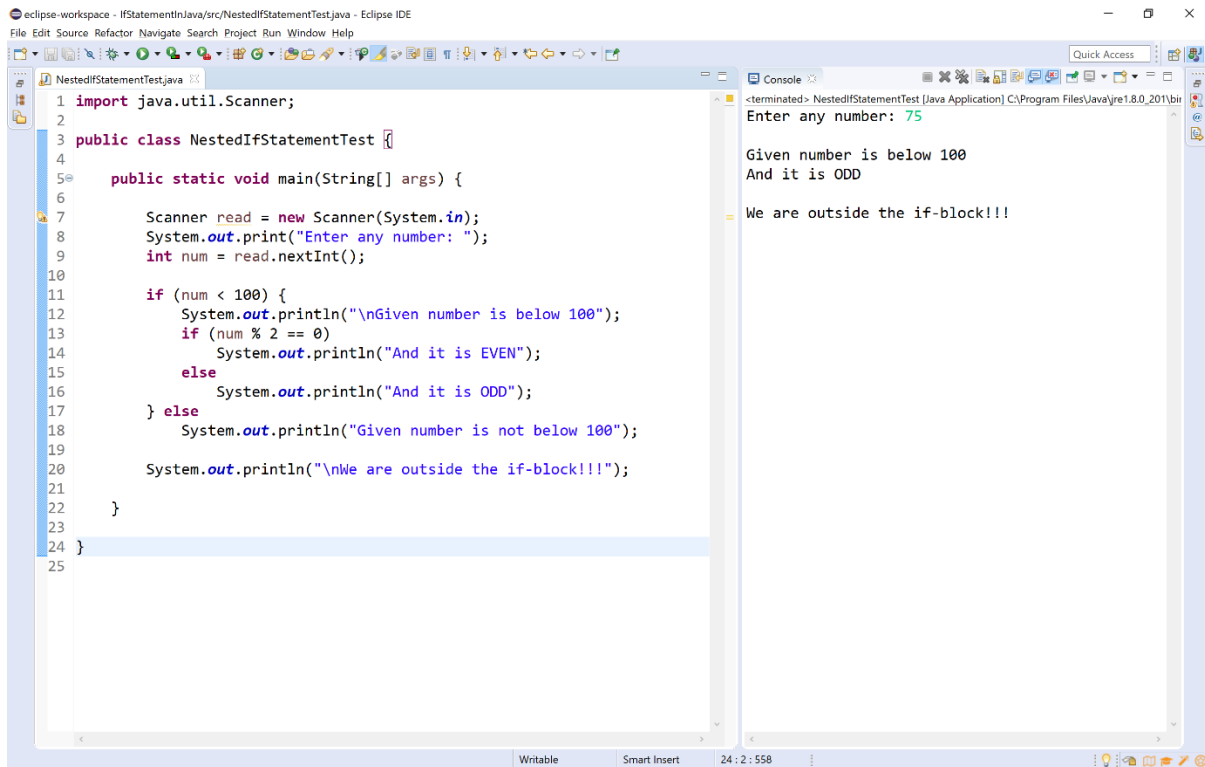
Nested if statement in java

Writing an if statement inside another if-statement is called nested if statement. The general syntax of the nested if-statement is as follows.

Syntax

```
if(condition_1){
    if(condition_2){
        inner if-block of statements;
        ...
    }
    ...
}
```

Let's look at the following example java code.



if-else if statement in java

Writing an if-statement inside else of an if statement is called if-else-if statement. The general syntax of the an if-else-if statement is as follows.

Syntax

```
if(condition_1){
    condition_1 true-block;

    ...
}
else if(condition_2){
    condition_2 true-block;
    condition_1 false-block too;

    ...
}
```

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java file named `IfElseIfStatementTest.java` and its console output.

```

1 import java.util.Scanner;
2
3 public class IfElseIfStatementTest {
4
5     public static void main(String[] args) {
6
7         int num1, num2, num3;
8         Scanner read = new Scanner(System.in);
9         System.out.print("Enter any three numbers: ");
10        num1 = read.nextInt();
11        num2 = read.nextInt();
12        num3 = read.nextInt();
13
14        if( num1>=num2 && num1>=num3)
15            System.out.println("\nThe largest number is " + num1) ;
16
17        else if ( num2>=num1 && num2>=num3)
18            System.out.println("\nThe largest number is " + num2) ;
19
20        else
21            System.out.println("\nThe largest number is " + num3) ;
22
23        System.out.println("\nWe are outside the if-block!!!");
24
25    }
26
27 }
28

```

The console output shows the program execution:

```

<terminated> NestedIfStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin
Enter any three numbers: 17 39 14

The largest number is 39

We are outside the if-block!!!

```

switch statement in java

Using the switch statement, one can select only one option from more number of options very easily. In the switch statement, we provide a value that is to be compared with a value associated with each option. Whenever the given value matches the value associated with an option, the execution starts from that option. In the switch statement, every option is defined as a **case**.

The switch statement has the following syntax and execution flow diagram.

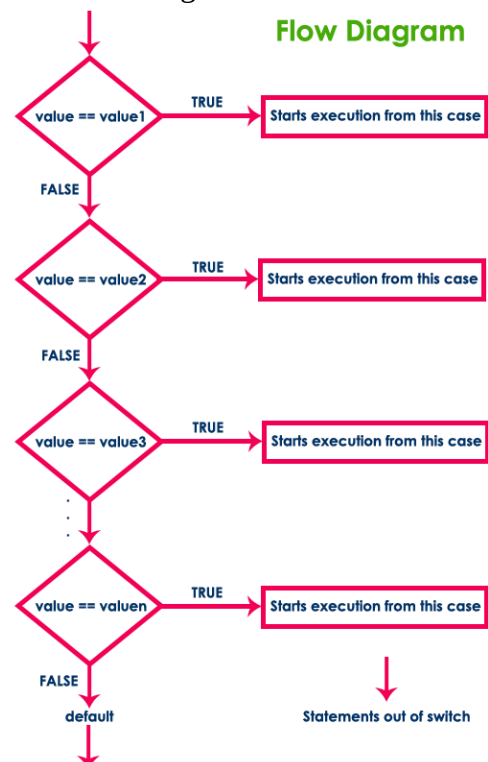
Syntax

```

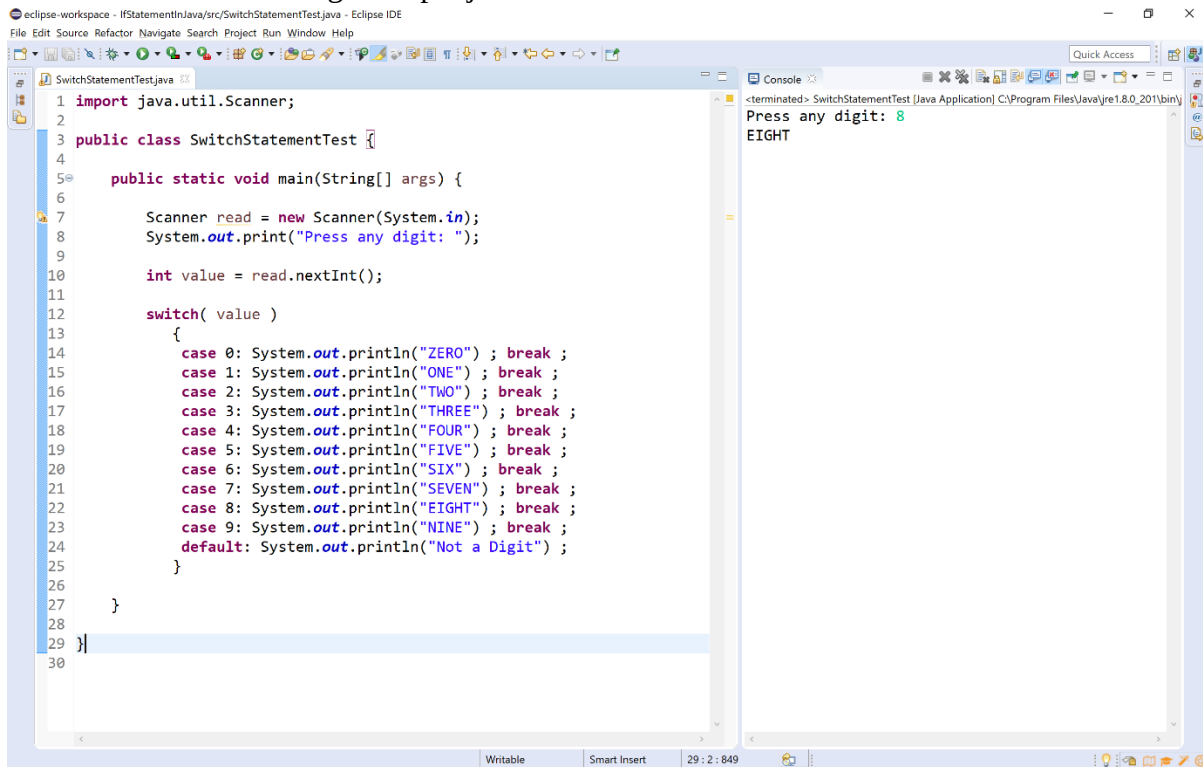
switch ( expression or value )
{
    case value1: set of statements;
        ....
    case value2: set of statements;
        ....
    case value3: set of statements;
        ....
    case value4: set of statements;
        ....
    case value5: set of statements;
        ....
    .
    .
    default: set of statements;
}

```

Flow Diagram



Let's look at the following example java code.



The screenshot shows the Eclipse IDE with a Java file named `SwitchStatementTest.java`. The code is as follows:

```
1 import java.util.Scanner;
2
3 public class SwitchStatementTest {
4
5     public static void main(String[] args) {
6
7         Scanner read = new Scanner(System.in);
8         System.out.print("Press any digit: ");
9
10        int value = read.nextInt();
11
12        switch( value )
13        {
14            case 0: System.out.println("ZERO") ; break ;
15            case 1: System.out.println("ONE") ; break ;
16            case 2: System.out.println("TWO") ; break ;
17            case 3: System.out.println("THREE") ; break ;
18            case 4: System.out.println("FOUR") ; break ;
19            case 5: System.out.println("FIVE") ; break ;
20            case 6: System.out.println("SIX") ; break ;
21            case 7: System.out.println("SEVEN") ; break ;
22            case 8: System.out.println("EIGHT") ; break ;
23            case 9: System.out.println("NINE") ; break ;
24            default: System.out.println("Not a Digit") ;
25        }
26
27    }
28
29 }
30
```

The console output shows the program execution:

```
<terminated> SwitchStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\
Press any digit: 8
EIGHT
```

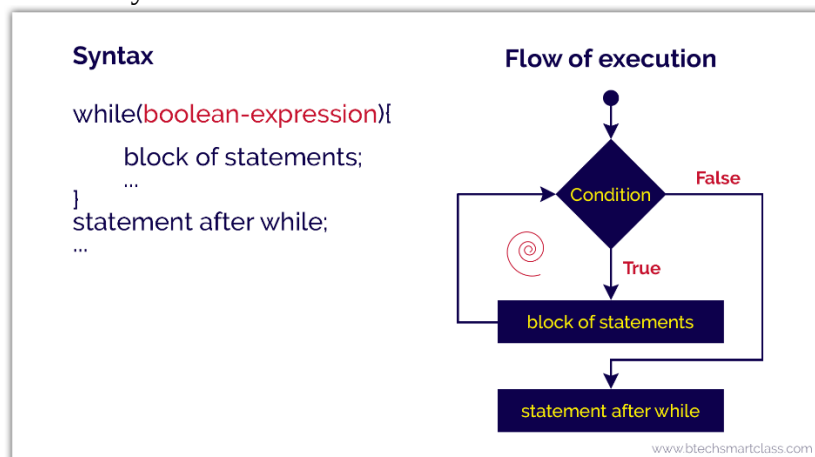
LOOPS

Java provides the following iterative statements.

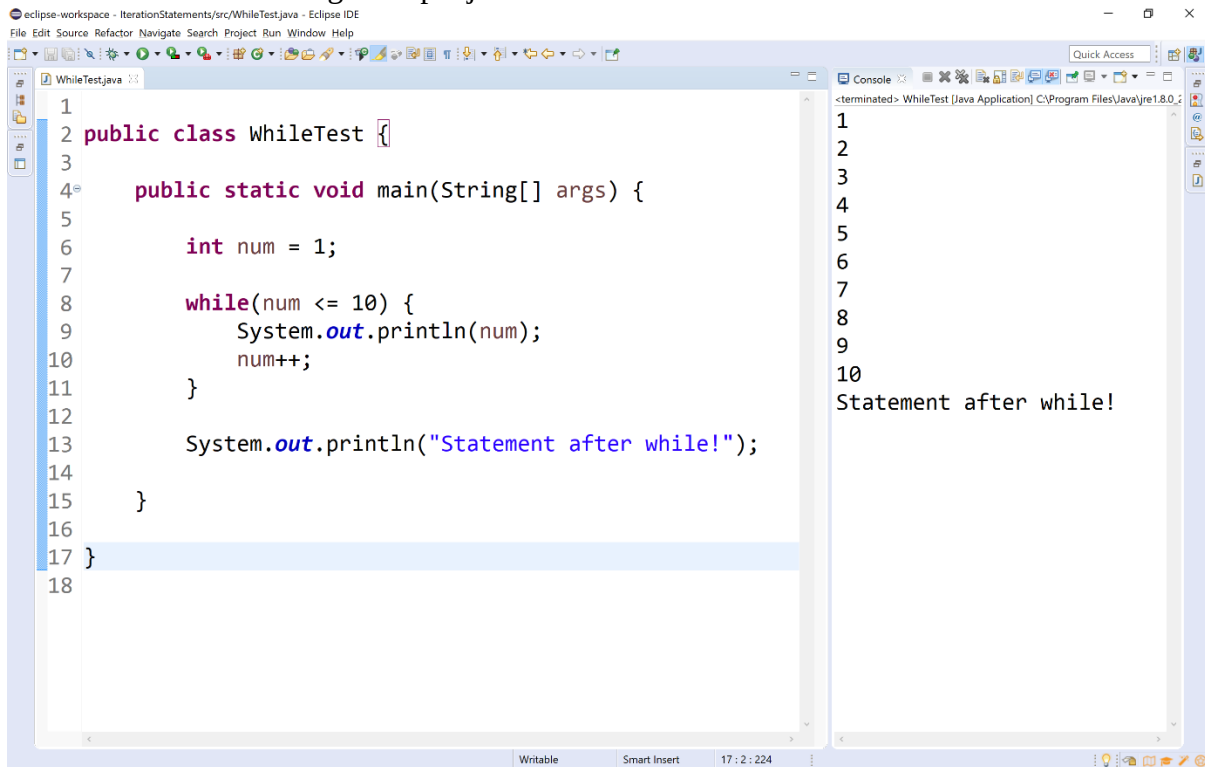
- while statement
- do-while statement
- for statement
- for-each statement

while statement in java

The while statement is used to execute a single statement or block of statements repeatedly as long as the given condition is TRUE. The while statement is also known as Entry control looping statement. The syntax and execution flow of while statement is as follows.



Let's look at the following example java code.



The screenshot shows the Eclipse IDE with a Java file named 'WhileTest.java'. The code is as follows:

```
1 public class WhileTest {  
2  
3     public static void main(String[] args) {  
4  
5         int num = 1;  
6  
7         while(num <= 10) {  
8             System.out.println(num);  
9             num++;  
10        }  
11  
12        System.out.println("Statement after while!");  
13  
14    }  
15 }  
16  
17  
18
```

The console on the right shows the output of the program:

```
<terminated> WhileTest [Java Application] C:\Program Files\Java\jre1.8.0_...  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
Statement after while!
```

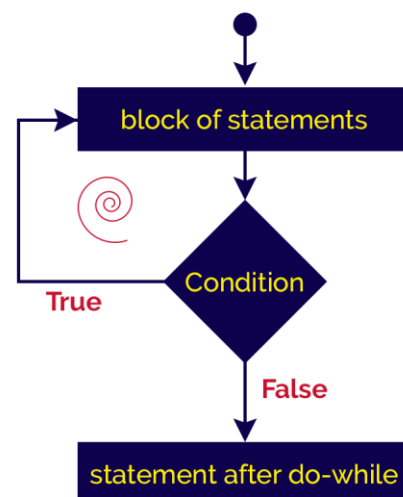
do-while statement in java

The do-while statement is used to execute a single statement or block of statements repeatedly as long as given the condition is TRUE. The do-while statement is also known as the **Exit control looping statement**. The do-while statement has the following syntax.

Syntax

```
do{  
    block of statements;  
}while(boolean-expression);  
statement after do-while;  
...
```

Flow of execution



Let's look at the following example java code.

```
eclipse-workspace - IterationStatements/src/DoWhileTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

DoWhileTest.java
1 public class DoWhileTest {
2
3
4     public static void main(String[] args) {
5
6         int num = 1;
7
8         do {
9             System.out.println(num);
10            num++;
11        }while(num <= 10);
12
13        System.out.println("Statement after do-while!");
14
15    }
16
17 }
18

Console
-terminated> DoWhileTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (
1
2
3
4
5
6
7
8
9
10
Statement after do-while!
```

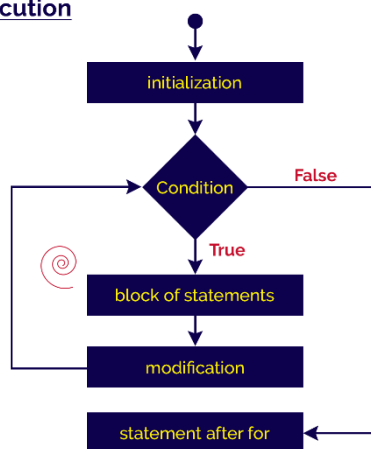
for statement in java

The for statement is used to execute a single statement or a block of statements repeatedly as long as the given condition is TRUE. The for statement has the following syntax and execution flow diagram.

Syntax

```
for(initialization; boolean-expression; modification){
    block of statements;
    ...
}
statement after for;
...
```

Flow of execution



In for-statement, the execution begins with the **initialization** statement. After the initialization statement, it executes **Condition**. If the condition is evaluated to true, then the block of statements executed otherwise it terminates the for-statement. After the block of statements execution, the **modification** statement gets executed, followed by condition again.

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a project named 'IterationStatements/src/ForTest.java'. The code in the editor is as follows:

```

1 public class ForTest {
2
3     public static void main(String[] args) {
4
5         for(int i = 0; i < 10; i++) {
6             System.out.println("i = " + i);
7         }
8
9         System.out.println("Statement after for!");
10    }
11 }
12
13
14

```

The console output on the right shows the execution results:

```

i = 0
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
Statement after for!

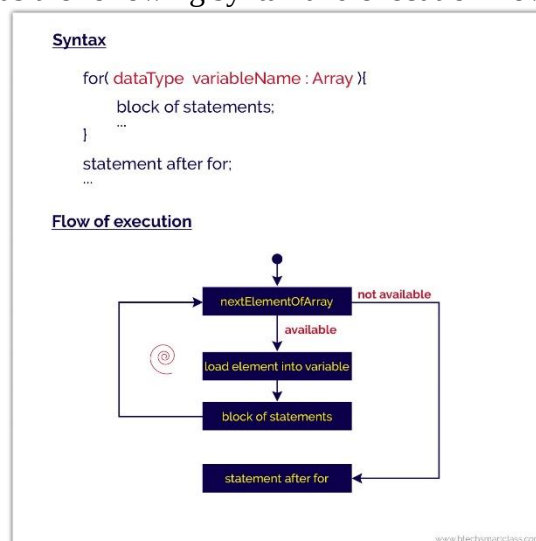
```

for-each statement in java

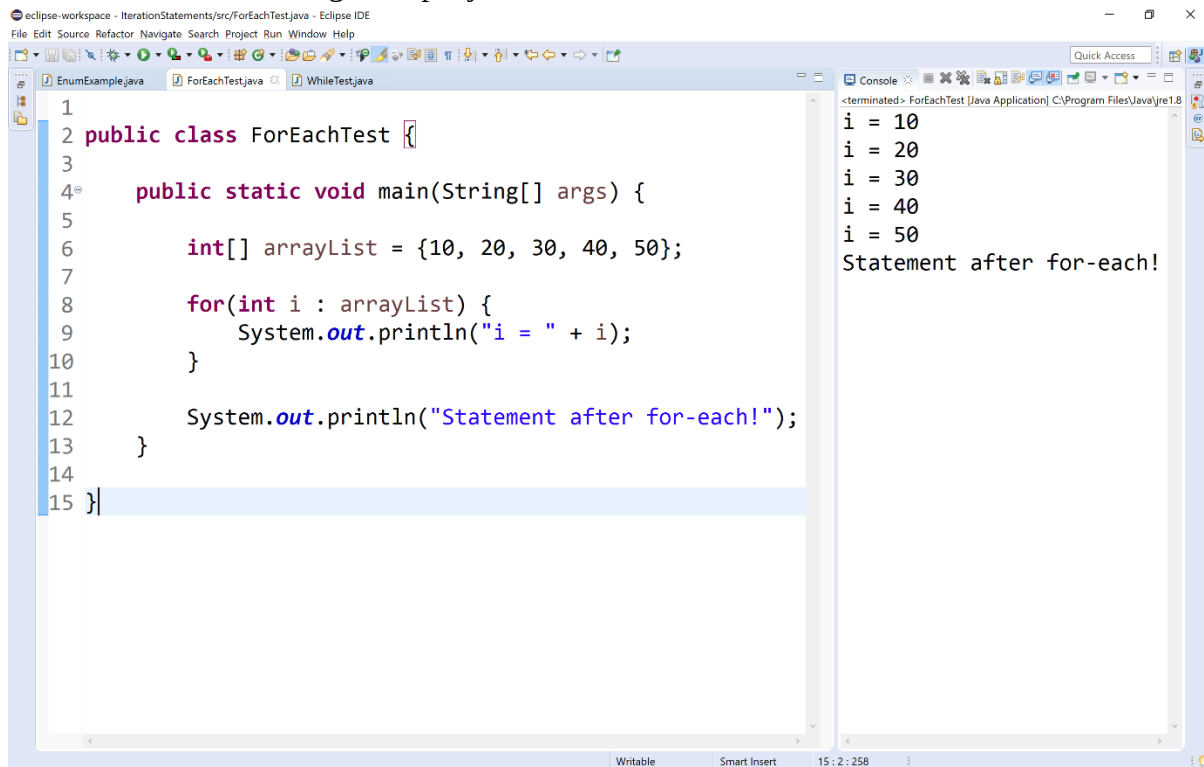
The Java for-each statement was introduced since Java 5.0 version. It provides an approach to traverse through an array or collection in Java. The for-each statement also known as **enhanced for** statement. The for-each statement executes the block of statements for each element of the given array or collection.

🔔 In for-each statement, we can not skip any element of given array or collection.

The for-each statement has the following syntax and execution flow diagram.



Let's look at the following example java code.



The screenshot shows the Eclipse IDE with a project named 'IterationStatements'. The 'ForEachTest.java' file is open, displaying the following code:

```
1
2 public class ForEachTest {
3
4     public static void main(String[] args) {
5
6         int[] arrayList = {10, 20, 30, 40, 50};
7
8         for(int i : arrayList) {
9             System.out.println("i = " + i);
10        }
11
12        System.out.println("Statement after for-each!");
13    }
14
15 }
```

The console on the right shows the output of the program:

```
<terminated> ForEachTest [Java Application] C:\Program Files\Java\jre1.8
i = 10
i = 20
i = 30
i = 40
i = 50
Statement after for-each!
```

type conversion and casting

1. Type Casting:

In typing casting, a data type is converted into another data type by the programmer using the casting operator during the program design. In typing casting, the destination data type may be smaller than the source data type when converting the data type to another data type, that's why it is also called narrowing conversion.

Syntax/Declaration:-

destination_datatype = (target_datatype)variable;

(): is a casting operator.

target_datatype: is a data type in which we want to convert the source data type.

Type Casting example –

float x;

byte y;

...

...

y=(byte)x; //Line 5

In Line 5: you can see that, we are converting **float(source)** data type into **byte(target)** data type.

2. Type conversion :

In type conversion, a data type is automatically converted into another data type by a compiler at the compiler time. In type conversion, the destination data type cannot be smaller than the source data type, that's why it is also called widening conversion. One more important thing is that it can only be applied to compatible data types.

Type Conversion example –

int x=30;

```
float y;  
y=x; // y==30.000000.
```

Let's see the **difference** between Type casting and Type conversion which are given below:

S.NO	TYPE CASTING	TYPE CONVERSION
1.	In type casting, a data type is converted into another data type by a programmer using casting operator.	Whereas in type conversion, a data type is converted into another data type by a compiler.
2.	Type casting can be applied to compatible data types as well as incompatible data types .	Whereas type conversion can only be applied to compatible datatypes .
3.	In type casting, casting operator is needed in order to cast the a data type to another data type.	Whereas in type conversion, there is no need for a casting operator.
4.	In typing casting, the destination data type may be smaller than the source data type, when converting the data type to another data type.	Whereas in type conversion, the destination data type can't be smaller than source data type.
5.	Type casting takes place during the program design by programmer.	Whereas type conversion is done at the compile time.
6.	Type casting is also called narrowing conversion because in this, the destination data type may be smaller than the source data type.	Whereas type conversion is also called widening conversion because in this, the destination data type can not be smaller than the source data type.
7.	Type casting is often used in coding and competitive programming works.	Whereas type conversion is less used in coding and competitive programming as it might cause incorrect answer.
8.	Type casting is more efficient and reliable.	Whereas type conversion is less efficient and less reliable.

concepts of classes, objects

In this chapter, we will look into the concepts - Classes and Objects.

- **Object** – Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behaviors – wagging the tail, barking, eating. An object is an instance of a class.
- **Class** – A class can be defined as a template/blueprint that describes the behavior/state that the object of its type support.

Objects in Java

Let us now look deep into what are objects. If we consider the real-world, we can find many objects around us, cars, dogs, humans, etc. All these objects have a state and a behavior.

If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging the tail, running.

If you compare the software object with a real-world object, they have very similar characteristics.

Software objects also have a state and a behavior. A software object's state is stored in fields and behavior is shown via methods.

So in software development, methods operate on the internal state of an object and the object-to-object communication is done via methods.

AD

Classes in Java

A class is a blueprint from which individual objects are created.

Following is a sample of a class.

Example

```
public class Dog {  
    String breed;  
    int age;  
    String color;  
  
    void barking() {  
    }  
  
    void hungry() {  
    }  
  
    void sleeping() {  
    }  
}
```

A class can contain any of the following variable types.

- **Local variables** – Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables** – Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.

- **Class variables** – Class variables are variables declared within a class, outside any method, with the static keyword.

A class can have any number of methods to access the value of various kinds of methods. In the above example, barking(), hungry() and sleeping() are methods.

Following are some of the important topics that need to be discussed when looking into classes of the Java Language.

Constructors

When discussing about classes, one of the most important sub topic would be constructors. Every class has a constructor. If we do not explicitly write a constructor for a class, the Java compiler builds a default constructor for that class.

Each time a new object is created, at least one constructor will be invoked. The main rule of constructors is that they should have the same name as the class. A class can have more than one constructor.

Following is an example of a constructor –

Example

```
public class Puppy {  
    public Puppy() {  
    }  
  
    public Puppy(String name) {  
        // This constructor has one parameter, name.  
    }  
}
```

Java also supports Singleton Classes where you would be able to create only one instance of a class.

Note – We have two different types of constructors. We are going to discuss constructors in detail in the subsequent chapters.

Creating an Object

As mentioned previously, a class provides the blueprints for objects. So basically, an object is created from a class. In Java, the new keyword is used to create new objects.

There are three steps when creating an object from a class –

- **Declaration** – A variable declaration with a variable name with an object type.
- **Instantiation** – The 'new' keyword is used to create the object.
- **Initialization** – The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

Following is an example of creating an object –

Example

```
public class Puppy {  
    public Puppy(String name) {  
        // This constructor has one parameter, name.  
        System.out.println("Passed Name is :" + name );  
    }  
}
```

```
public static void main(String []args) {  
    // Following statement would create an object myPuppy  
    Puppy myPuppy = new Puppy( "tommy" );  
}  
}
```

If we compile and run the above program, then it will produce the following result –

Output

Passed Name is :tommy

Accessing Instance Variables and Methods

Instance variables and methods are accessed via created objects. To access an instance variable, following is the fully qualified path –

/* First create an object */

ObjectReference = new Constructor();

/* Now call a variable as follows */

ObjectReference.variableName;

/* Now you can call a class method as follows */

ObjectReference.MethodName();

Example

This example explains how to access instance variables and methods of a class.

```
public class Puppy {  
    int puppyAge;  
  
    public Puppy(String name) {  
        // This constructor has one parameter, name.  
        System.out.println("Name chosen is :" + name );  
    }  
  
    public void setAge( int age ) {  
        puppyAge = age;  
    }  
  
    public int getAge( ) {  
        System.out.println("Puppy's age is :" + puppyAge );  
        return puppyAge;  
    }  
  
    public static void main(String []args) {  
        /* Object creation */  
        Puppy myPuppy = new Puppy( "tommy" );  
  
        /* Call class method to set puppy's age */  
        myPuppy.setAge( 2 );  
  
        /* Call another class method to get puppy's age */  
        myPuppy.getAge( );  
    }  
}
```

```
/* You can access instance variable as follows as well */  
System.out.println("Variable Value :" + myPuppy.puppyAge );  
}  
}
```

If we compile and run the above program, then it will produce the following result –

Output

Name chosen is :tommy

Puppy's age is :2

Variable Value :2

Constructors

A constructor initializes an object when it is created. It has the same name as its class and is syntactically similar to a method. However, constructors have no explicit return type.

Typically, you will use a constructor to give initial values to the instance variables defined by the class, or to perform any other start-up procedures required to create a fully formed object.

All classes have constructors, whether you define one or not, because Java automatically provides a default constructor that initializes all member variables to zero. However, once you define your own constructor, the default constructor is no longer used.

Syntax

Following is the syntax of a constructor –

```
class ClassName {  
    ClassName() {  
    }  
}
```

Java allows two types of constructors namely –

- No argument Constructors
- Parameterized Constructors

No argument Constructors

As the name specifies the no argument constructors of Java does not accept any parameters instead, using these constructors the instance variables of a method will be initialized with fixed values for all objects.

Example

```
Public class MyClass {  
    Int num;  
    MyClass() {  
        num = 100;  
    }  
}
```

You would call constructor to initialize objects as follows

```
public class ConsDemo {  
    public static void main(String args[]) {  
        MyClass t1 = new MyClass();  
        MyClass t2 = new MyClass();  
        System.out.println(t1.num + " " + t2.num);  
    }  
}
```

This would produce the following result
100 100

Parameterized Constructors

Most often, you will need a constructor that accepts one or more parameters. Parameters are added to a constructor in the same way that they are added to a method, just declare them inside the parentheses after the constructor's name.

Example

Here is a simple example that uses a constructor –

```
// A simple constructor.  
class MyClass {  
    int x;  
  
    // Following is the constructor  
    MyClass(int i ) {  
        x = i;  
    }  
}
```

You would call constructor to initialize objects as follows –

```
public class ConsDemo {  
    public static void main(String args[]) {  
        MyClass t1 = new MyClass( 10 );  
        MyClass t2 = new MyClass( 20 );  
        System.out.println(t1.x + " " + t2.x);  
    }  
}
```

This would produce the following result –

10 20

Methods

A method is a block of statements under a name that gets executed only when it is called. Every method is used to perform a specific task. The major advantage of methods is code re-usability (define the code once, and use it many times).

In a java programming language, a method is defined as a behavior of an object. That means, every method in java must belong to a class.

Every method in java must be declared inside a class.

Every method declaration has the following characteristics.

- **returnType** - Specifies the data type of a return value.
- **name** - Specifies a unique name to identify it.
- **parameters** - The data values it may accept or receive.
- **{ }** - Defines the block belongs to the method.

Creating a method

A method is created inside the class and it may be created with any access specifier. However, specifying access specifier is optional.

Following is the syntax for creating methods in java.

Syntax

```
class <ClassName>{
    <accessSpecifier> <returnType> <methodName>( parameters ){
        ...
        block of statements;
        ...
    }
}
```

- 🔔 The methodName must begin with an alphabet, and the Lower-case letter is preferred.
- 🔔 The methodName must follow all naming rules.
- 🔔 If you don't want to pass parameters, we ignore it.
- 🔔 If a method defined with return type other than void, it must contain the return statement, otherwise, it may be ignored.

Calling a method

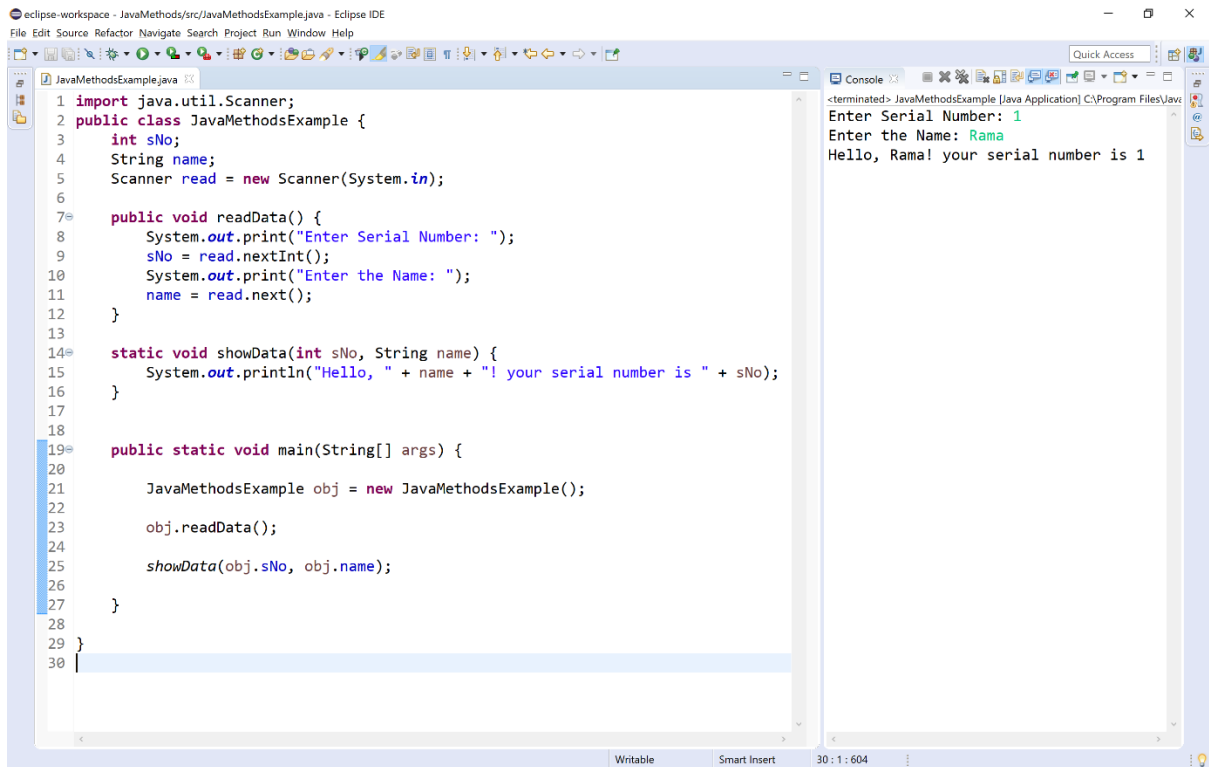
In java, a method call precedes with the object name of the class to which it belongs and a dot operator. It may call directly if the method defined with the static modifier. Every method call must be made, as to the method name with parentheses (), and it must terminate with a semicolon.

Syntax

```
<objectName>.<methodName>( actualArguments );
```

- 🔔 The method call must pass the values to parameters if it has.
- 🔔 If the method has a return type, we must provide the receiver.

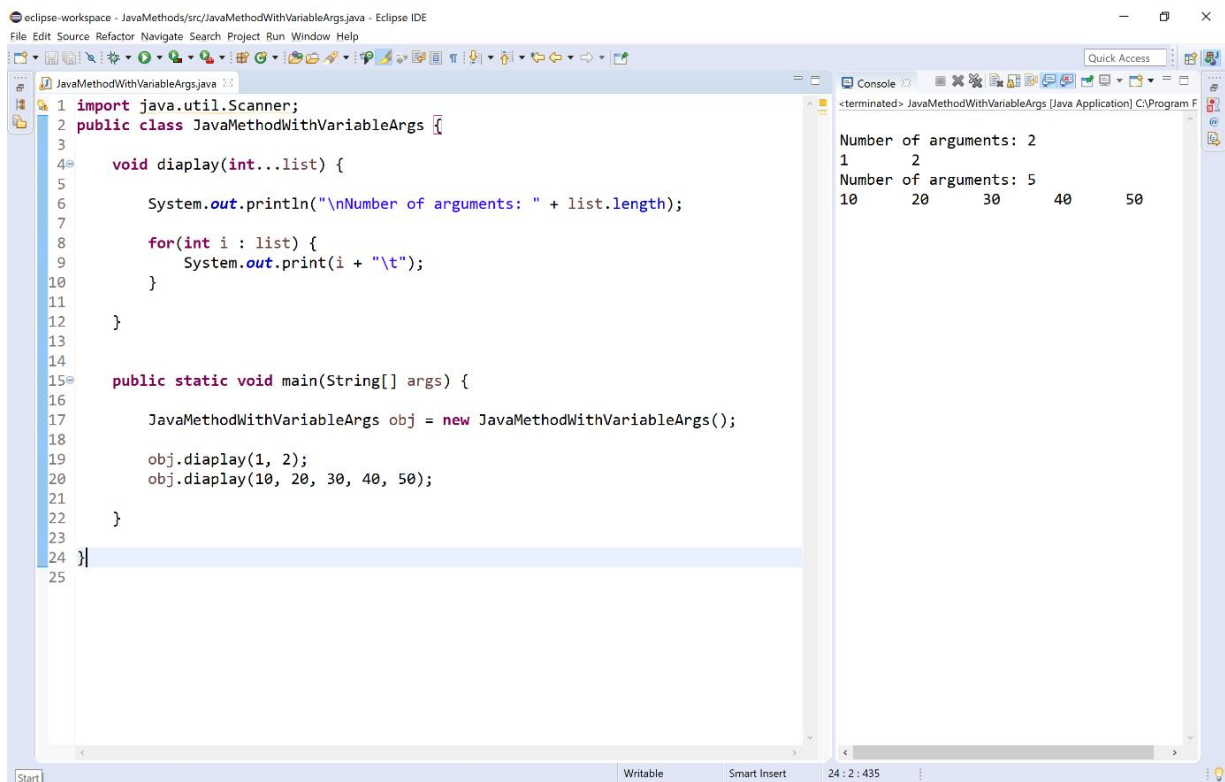
Let's look at the following example java code.



Variable arguments of a method

In java, a method can be defined with a variable number of arguments. That means creating a method that receives any number of arguments of the same data type. **Syntax**

`<returnType> <methodName>(dataType...parameterName);`



access control

In Java, the access specifiers (also known as access modifiers) used to restrict the scope or accessibility of a class, constructor, variable, method or data member of class and interface. There are four access specifiers, and their list is below.

- **default (or) no modifier**
- **public**
- **protected**
- **private**

In java, we can not employ all access specifiers on everything. The following table describes where we can apply the access specifiers.

Access Specifier Item	Default	Public	Protected	Private
Class	Yes	Yes	No	No
Inner Class	Yes	Yes	Yes	Yes
Interface	Yes	Yes	No	No
Interface Inside Class	Yes	Yes	Yes	Yes
enum	Yes	Yes	No	No
enum Inside Class	Yes	Yes	Yes	Yes
enum inside Interface	Yes	No	No	No
Constructor	Yes	Yes	Yes	Yes
methods & data inside class	Yes	Yes	Yes	Yes
methods & data inside Interface	Yes	No	No	No

www.btechsmartclass.com

Let's look at the following example java code, which generates an error because a class does not allow private access specifier unless it is an inner class.

Example

```
private class Sample{  
    ...  
}
```

In java, the accessibility of the members of a class or interface depends on its access specifiers. The following table provides information about the visibility of both data members and methods.

Access control for members of class and interface in java

Access Specifier \ Accessibility Location	Same Class	Same Package		Other Package	
		Child class	Non-child class	Child class	Non-child class
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No

www.btechsmartclass.com

- 🔔 The **public** members can be accessed everywhere.
- 🔔 The **private** members can be accessed only inside the same class.
- 🔔 The **protected** members are accessible to every child class (same package or other packages).
- 🔔 The **default** members are accessible within the same package but not outside the package.

Let's look at the following example java code.

```

1 class ParentClass{
2     int a = 10;
3     public int b = 20;
4     protected int c = 30;
5     private int d = 40;
6
7     void showData() {
8         System.out.println("Inside ParentClass");
9         System.out.println("a = " + a);
10        System.out.println("b = " + b);
11        System.out.println("c = " + c);
12        System.out.println("d = " + d);
13    }
14 }
15
16 class ChildClass extends ParentClass{
17
18     void accessData() {
19         System.out.println("Inside ChildClass");
20         System.out.println("a = " + a);
21         System.out.println("b = " + b);
22         System.out.println("c = " + c);
23         //System.out.println("d = " + d); // private member can't be accessed
24     }
25 }
26
27 public class AccessModifiersExample {
28
29     public static void main(String[] args) {
30
31         ChildClass obj = new ChildClass();
32         obj.showData();
33         obj.accessData();
34     }
35 }

```

Console Output:

```

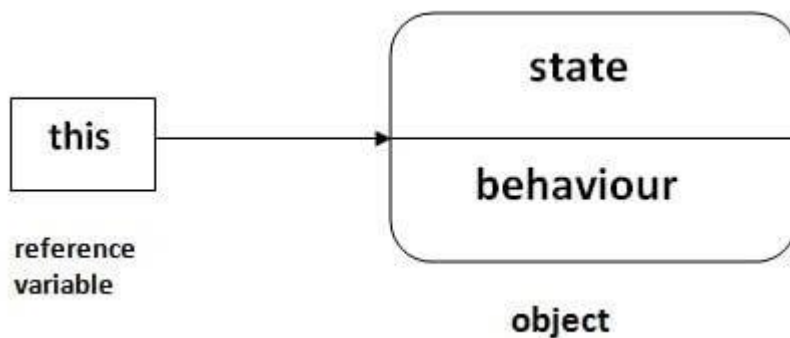
<terminated> AccessModifiersExample [Java Application] C:\Progra
Inside ParentClass
a = 10
b = 20
c = 30
d = 40
Inside ChildClass
a = 10
b = 20
c = 30

```

this keyword

this keyword in Java

There can be a lot of usage of **Java this keyword**. In Java, this is a **reference variable** that refers to the current object.

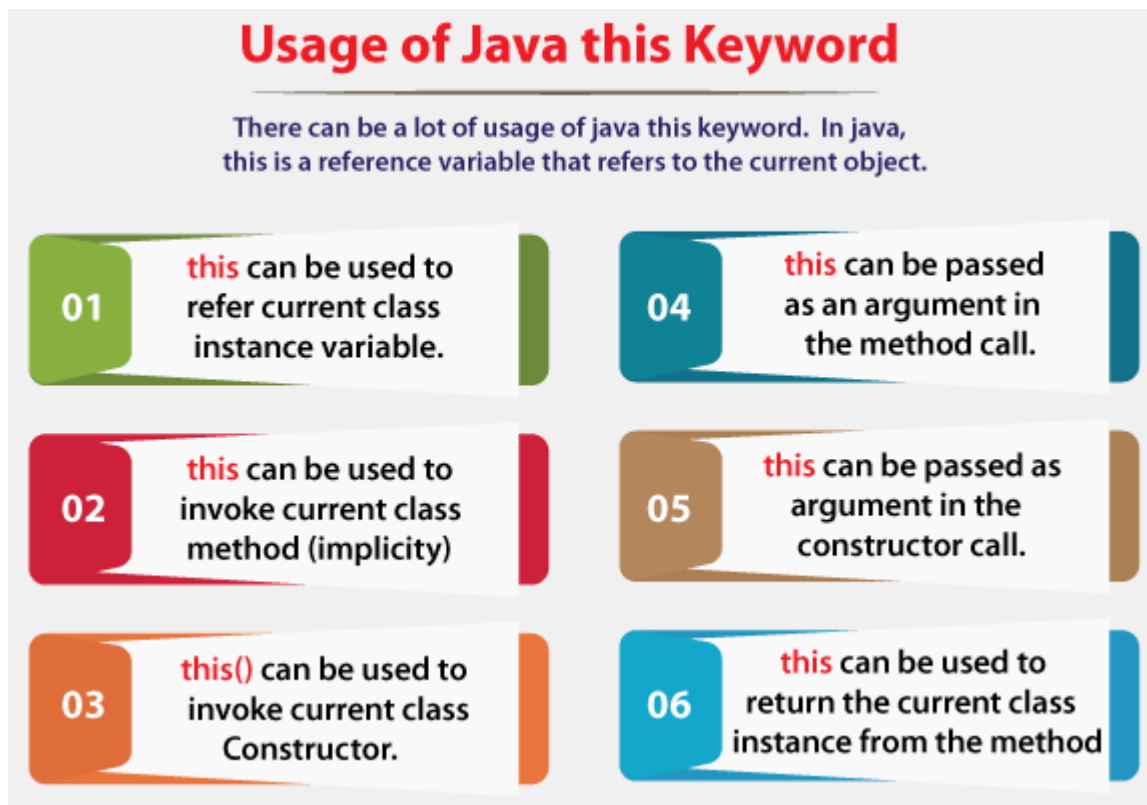


Usage of Java this keyword

Here is given the 6 usage of java this keyword.

1. this can be used to refer current class instance variable.
2. this can be used to invoke current class method (implicitly)
3. this() can be used to invoke current class constructor.
4. this can be passed as an argument in the method call.
5. this can be passed as argument in the constructor call.

6. this can be used to return the current class instance from the method.



1) **this**: to refer current class instance variable

The this keyword can be used to refer current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

Understanding the problem without this keyword

Let's understand the problem if we don't use this keyword by the example given below:

1. **class** Student{
2. **int** rollno;
3. String name;
4. **float** fee;
5. Student(**int** rollno,String name,**float** fee){
6. rollno=rollno;
7. name=name;
8. fee=fee;
9. }
10. **void** display(){System.out.println(rollno+" "+name+" "+fee);}
11. }

```

12. class TestThis1{
13. public static void main(String args[]){
14. Student s1=new Student(111,"ankit",5000f);
15. Student s2=new Student(112,"sumit",6000f);
16. s1.display();
17. s2.display();
18. }}

```

Output:

```

0 null 0.0
0 null 0.0

```

In the above example, parameters (formal arguments) and instance variables are same. So, we are using this keyword to distinguish local variable and instance variable.

Solution of the above problem by this keyword

```

1. class Student{
2. int rollno;
3. String name;
4. float fee;
5. Student(int rollno,String name,float fee){
6. this.rollno=rollno;
7. this.name=name;
8. this.fee=fee;
9. }
10. void display(){System.out.println(rollno+" "+name+" "+fee);}
11. }
12.
13. class TestThis2{
14. public static void main(String args[]){
15. Student s1=new Student(111,"ankit",5000f);
16. Student s2=new Student(112,"sumit",6000f);
17. s1.display();
18. s2.display();
19. }}

```

Output:

```

111 ankit 5000.0
112 sumit 6000.0

```

If local variables(formal arguments) and instance variables are different, there is no need to use this keyword like in the following program:

Program where this keyword is not required

```
1. class Student{
2.   int rollno;
3.   String name;
4.   float fee;
5.   Student(int r,String n,float f){
6.     rollno=r;
7.     name=n;
8.     fee=f;
9.   }
10. void display(){System.out.println(rollno+" "+name+" "+fee);}
11. }
12.
13. class TestThis3{
14.   public static void main(String args[]){
15.     Student s1=new Student(111,"ankit",5000f);
16.     Student s2=new Student(112,"sumit",6000f);
17.     s1.display();
18.     s2.display();
19. }}
```

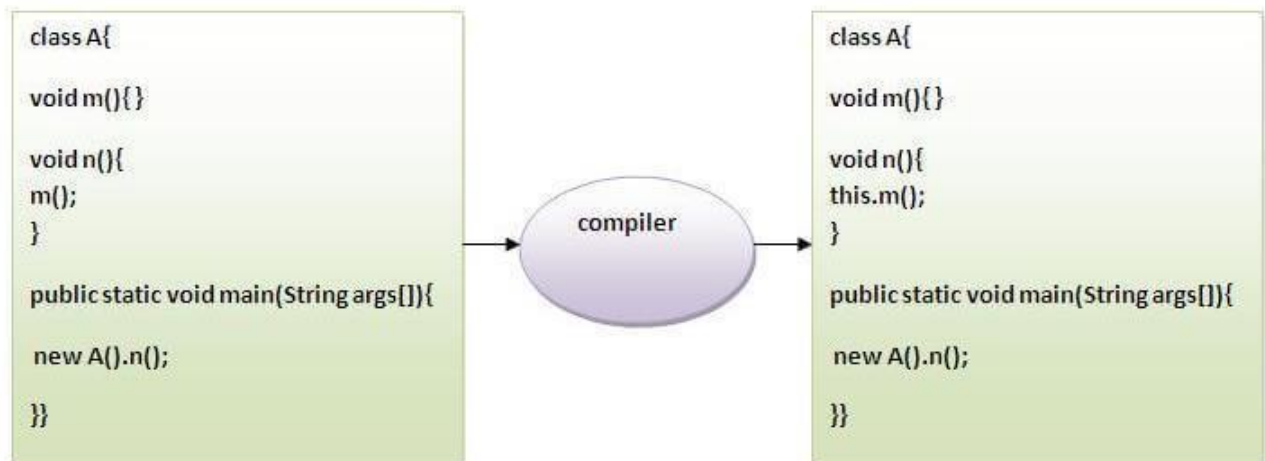
Output:

```
111 ankit 5000.0
112 sumit 6000.0
```

It is better approach to use meaningful names for variables. So we use same name for instance variables and parameters in real time, and always use this keyword.

2) this: to invoke current class method

You may invoke the method of the current class by using the this keyword. If you don't use the this keyword, compiler automatically adds this keyword while invoking the method. Let's see the example



1. **class** A{
2. **void** m(){System.out.println("hello m");}
3. **void** n(){
4. System.out.println("hello n");
5. //m();//same as this.m()
6. **this.m()**;
7. }
8. }
9. **class** TestThis4{
10. **public static void** main(String args[]){
11. A a=**new** A();
12. a.n();
13. }}

Output:

```
hello n
hello m
```

3) this() : to invoke current class constructor

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

Calling default constructor from parameterized constructor:

1. **class** A{
2. A(){System.out.println("hello a");}
3. A(**int** x){
4. **this()**;
5. System.out.println(x);

```
6. }
7. }
8. class TestThis5{
9. public static void main(String args[]){
10. A a=new A(10);
11. }}
```

Output:

```
hello a
10
```

Calling parameterized constructor from default constructor:

```
1. class A{
2. A(){
3. this(5);
4. System.out.println("hello a");
5. }
6. A(int x){
7. System.out.println(x);
8. }
9. }
10. class TestThis6{
11. public static void main(String args[]){
12. A a=new A();
13. }}
```

Output:

```
5
hello a
```

Real usage of this() constructor call

The this() constructor call should be used to reuse the constructor from the constructor. It maintains the chain between the constructors i.e. it is used for constructor chaining. Let's see the example given below that displays the actual use of this keyword.

```
1. class Student{
2. int rollno;
3. String name,course;
4. float fee;
5. Student(int rollno,String name,String course){
```

```

6.  this.rollno=rollno;
7.  this.name=name;
8.  this.course=course;
9.  }
10. Student(int rollno,String name,String course,float fee){
11. this(rollno,name,course);//reusing constructor
12. this.fee=fee;
13. }
14. void display(){System.out.println(rollno+" "+name+" "+course+" "+fee);}
15. }
16. class TestThis7{
17. public static void main(String args[]){
18. Student s1=new Student(111,"ankit","java");
19. Student s2=new Student(112,"sumit","java",6000f);
20. s1.display();
21. s2.display();
22. }}

```

Output:

```

111 ankit java 0.0
112 sumit java 6000.0

```

Rule: Call to *this()* must be the first statement in constructor.

```

1. class Student{
2.  int rollno;
3.  String name,course;
4.  float fee;
5.  Student(int rollno,String name,String course){
6.  this.rollno=rollno;
7.  this.name=name;
8.  this.course=course;
9.  }
10. Student(int rollno,String name,String course,float fee){
11. this.fee=fee;
12. this(rollno,name,course);//C.T.Error
13. }
14. void display(){System.out.println(rollno+" "+name+" "+course+" "+fee);}
15. }

```



```
16. class TestThis8{
17. public static void main(String args[]){
18. Student s1=new Student(111,"ankit","java");
19. Student s2=new Student(112,"sumit","java",6000f);
20. s1.display();
21. s2.display();
22. }}
```

Output:

```
Compile Time Error: Call to this must be first statement in constructor
```

4) this: to pass as an argument in the method

The this keyword can also be passed as an argument in the method. It is mainly used in the event handling. Let's see the example:

```
1. class S2{
2. void m(S2 obj){
3. System.out.println("method is invoked");
4. }
5. void p(){
6. m(this);
7. }
8. public static void main(String args[]){
9. S2 s1 = new S2();
10. s1.p();
11. }
12. }
```

Output:

```
method is invoked
```

Application of this that can be passed as an argument:

In event handling (or) in a situation where we have to provide reference of a class to another one. It is used to reuse one object in many methods.

5) this: to pass as argument in the constructor call

We can pass the this keyword in the constructor also. It is useful if we have to use one object in multiple classes. Let's see the example:

```

1. class B{
2.     A4 obj;
3.     B(A4 obj){
4.         this.obj=obj;
5.     }
6.     void display(){
7.         System.out.println(obj.data);//using data member of A4 class
8.     }
9. }
10.
11. class A4{
12.     int data=10;
13.     A4(){
14.         B b=new B(this);
15.         b.display();
16.     }
17.     public static void main(String args[]){
18.         A4 a=new A4();
19.     }
20. }

```

Output:10

6) this keyword can be used to return current class instance

We can return this keyword as an statement from the method. In such case, return type of the method must be the class type (non-primitive). Let's see the example:

Syntax of this that can be returned as a statement

```

1. return_type method_name(){
2.     return this;
3. }

```

Example of this keyword that you return as a statement from the method

```

1. class A{
2.     A getA(){
3.         return this;
4.     }
5.     void msg(){System.out.println("Hello java");}
6. }

```

```
7. class Test1{
8. public static void main(String args[]){
9. new A().getA().msg();
10. }
11. }
```

Output:

```
Hello java
```

Proving this keyword

Let's prove that this keyword refers to the current class instance variable. In this program, we are printing the reference variable and this, output of both variables are same.

```
1. class A5{
2. void m(){
3. System.out.println(this); //prints same reference ID
4. }
5. public static void main(String args[]){
6. A5 obj=new A5();
7. System.out.println(obj); //prints the reference ID
8. obj.m();
9. }
10. }
```

Output:

```
A5@22b3ea59
A5@22b3ea59
```

garbage collection

In java, garbage means unreferenced objects.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.

To do so, we were using free() function in C language and delete() in C++. But, in java it is performed automatically. So, java provides better memory management.

Advantage of Garbage Collection

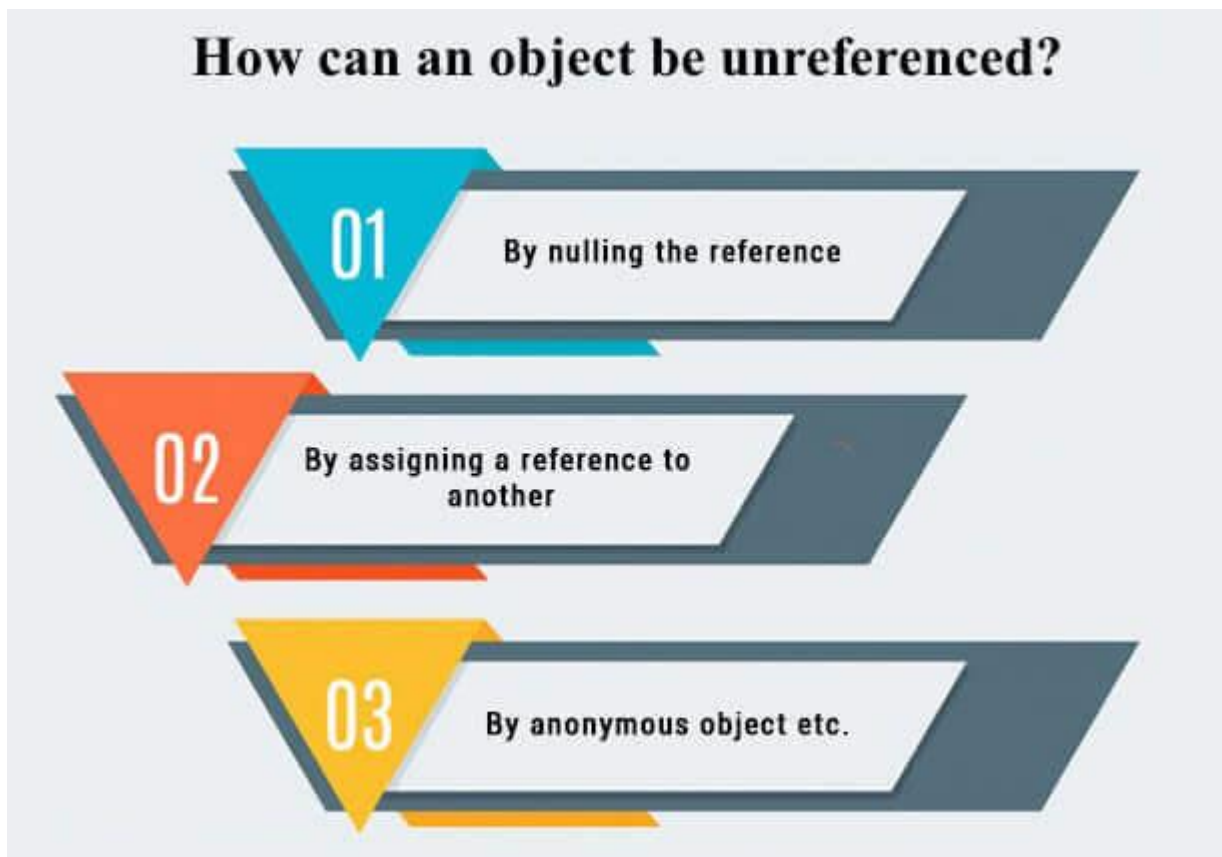
- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.

- It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extra efforts.
-

How can an object be unreferenced?

There are many ways:

- By nulling the reference
- By assigning a reference to another
- By anonymous object etc.



1) By nulling a reference:

1. Employee e=**new** Employee();
2. e=**null**;

2) By assigning a reference to another:

1. Employee e1=**new** Employee();
2. Employee e2=**new** Employee();
3. e1=e2;//now the first object referred by e1 is available for garbage collection

3) By anonymous object:

1. **new** Employee();
-

finalize() method

The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing. This method is defined in Object class as:

1. **protected void finalize(){}**

Note: The Garbage collector of JVM collects only those objects that are created by new keyword. So if you have created any object without new, you can use finalize method to perform cleanup processing (destroying remaining objects).

gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

1. **public static void gc(){}**

Note: Garbage collection is performed by a daemon thread called Garbage Collector(GC). This thread calls the finalize() method before object is garbage collected.

Simple Example of garbage collection in java

1. **public class** TestGarbage1{
2. **public void** finalize(){System.out.println("object is garbage collected");}
3. **public static void** main(String args[]){
4. TestGarbage1 s1=**new** TestGarbage1();
5. TestGarbage1 s2=**new** TestGarbage1();
6. s1=**null**;
7. s2=**null**;
8. System.gc();
9. }
10. }

```
object is garbage collected
object is garbage collected
```

overloading methods and constructors

Method Overloading

When a class has two or more methods by the same name but different parameters, it is known as method overloading. It is different from overriding. In overriding, a method has the same method name, type, number of parameters, etc.

Let's consider the example discussed earlier for finding minimum numbers of integer type. If, let's say we want to find the minimum number of double type. Then the concept of overloading will be introduced to create two or more methods with the same name but different parameters.

The following example explains the same –

Example

Live Demo

```
public class ExampleOverloading {

    public static void main(String[] args) {
        int a = 11;
        int b = 6;
        double c = 7.3;
        double d = 9.4;
        int result1 = minFunction(a, b);

        // same function name with different parameters
        double result2 = minFunction(c, d);
        System.out.println("Minimum Value = " + result1);
        System.out.println("Minimum Value = " + result2);
    }

    // for integer
    public static int minFunction(int n1, int n2) {
        int min;
        if (n1 > n2)
            min = n2;
        else
            min = n1;

        return min;
    }

    // for double
    public static double minFunction(double n1, double n2) {
        double min;
        if (n1 > n2)
            min = n2;
        else
            min = n1;

        return min;
    }
}
```

This will produce the following result –

Output

Minimum Value = 6

Minimum Value = 7.3

Overloading methods makes program readable. Here, two methods are given by the same name but with different parameters. The minimum number from integer and double types is the result.

Constructor Overloading in Java

In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example of Constructor Overloading

```
1. //Java program to overload constructors
2. class Student5{
3.     int id;
4.     String name;
5.     int age;
6.     //creating two arg constructor
7.     Student5(int i,String n){
8.         id = i;
9.         name = n;
10.    }
11.    //creating three arg constructor
12.    Student5(int i,String n,int a){
13.        id = i;
14.        name = n;
15.        age=a;
16.    }
17.    void display(){System.out.println(id+" "+name+" "+age);}
18.
19.    public static void main(String args[]){
20.        Student5 s1 = new Student5(111,"Karan");
21.        Student5 s2 = new Student5(222,"Aryan",25);
22.        s1.display();
23.        s2.display();
24.    }
25. }
```

Output:

```
111 Karan 0
222 Aryan 25
```

Introducing access control

In Java, the access specifiers (also known as access modifiers) used to restrict the scope or accessibility of a class, constructor, variable, method or data member of class and interface. There are four access specifiers, and their list is below.

- **default (or) no modifier**
- **public**
- **protected**
- **private**

In java, we can not employ all access specifiers on everything. The following table describes where we can apply the access specifiers.

Access Specifier Item	Default	Public	Protected	Private
Class	Yes	Yes	No	No
Inner Class	Yes	Yes	Yes	Yes
Interface	Yes	Yes	No	No
Interface Inside Class	Yes	Yes	Yes	Yes
enum	Yes	Yes	No	No
enum Inside Class	Yes	Yes	Yes	Yes
enum inside Interface	Yes	No	No	No
Constructor	Yes	Yes	Yes	Yes
methods & data inside class	Yes	Yes	Yes	Yes
methods & data inside Interface	Yes	No	No	No

www.btechsmartclass.com

Let's look at the following example java code, which generates an error because a class does not allow private access specifier unless it is an inner class.

Example


```
private class Sample{
    ...
}
```

In java, the accessibility of the members of a class or interface depends on its access specifiers. The following table provides information about the visibility of both data members and methods.

Access control for members of class and interface in java

Access Specifier \ Accessibility Location	Same Class	Same Package		Other Package	
		Child class	Non-child class	Child class	Non-child class
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No

www.btechsmartclass.com

- 🔔 The **public** members can be accessed everywhere.
- 🔔 The **private** members can be accessed only inside the same class.
- 🔔 The **protected** members are accessible to every child class (same package or other packages).
- 🔔 The **default** members are accessible within the same package but not outside the package.

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java project named 'Inheritance'. The main editor displays the file 'AccessModifiersExample.java'. The code defines a 'ParentClass' with attributes 'a' (int), 'b' (public int), 'c' (protected int), and 'd' (private int). It has a 'showData()' method that prints the values of these attributes. A 'ChildClass' extends 'ParentClass' and has an 'accessData()' method that prints the values of 'a', 'b', and 'c', but it has a commented-out line for 'd' with a note: '// private member can't be accessed'. The 'AccessModifiersExample' class contains a 'main' method that creates an instance of 'ChildClass' and calls both 'showData()' and 'accessData()' methods. The console on the right shows the output of the program, which is divided into two sections: 'Inside ParentClass' and 'Inside ChildClass'. The output for 'ParentClass' shows 'a = 10', 'b = 20', 'c = 30', and 'd = 40'. The output for 'ChildClass' shows 'a = 10', 'b = 20', and 'c = 30', but it does not show 'd'.

```
1 class ParentClass{
2     int a = 10;
3     public int b = 20;
4     protected int c = 30;
5     private int d = 40;
6
7     void showData() {
8         System.out.println("Inside ParentClass");
9         System.out.println("a = " + a);
10        System.out.println("b = " + b);
11        System.out.println("c = " + c);
12        System.out.println("d = " + d);
13    }
14 }
15
16 class ChildClass extends ParentClass{
17
18     void accessData() {
19         System.out.println("Inside ChildClass");
20         System.out.println("a = " + a);
21         System.out.println("b = " + b);
22         System.out.println("c = " + c);
23         //System.out.println("d = " + d); // private member can't be accessed
24     }
25 }
26
27 public class AccessModifiersExample {
28
29     public static void main(String[] args) {
30
31         ChildClass obj = new ChildClass();
32         obj.showData();
33         obj.accessData();
34     }
35 }
```

Console Output:

```
<terminated> AccessModifiersExample [Java Application] C:\Progra
Inside ParentClass
a = 10
b = 20
c = 30
d = 40
Inside ChildClass
a = 10
b = 20
c = 30
```

Static

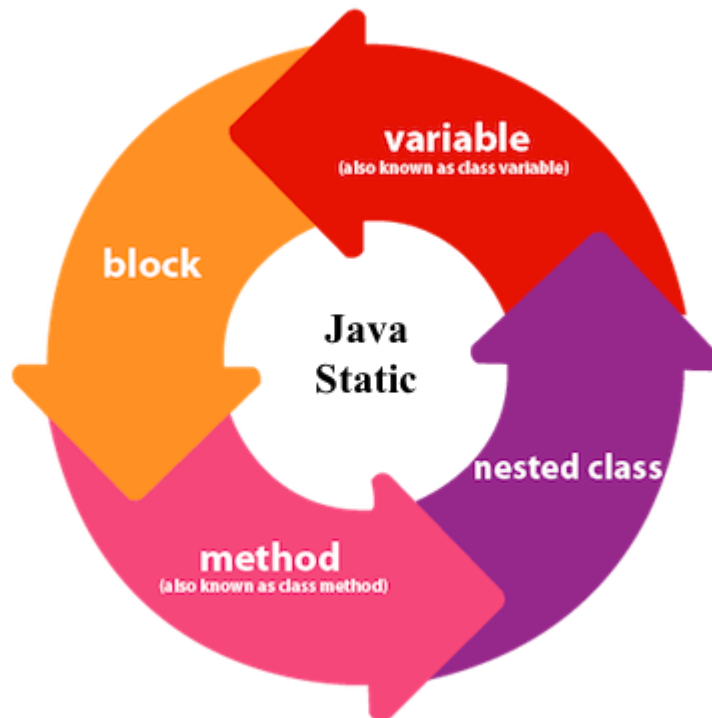
Java static keyword

1. Static variable
2. Program of the counter without static variable
3. Program of the counter with static variable
4. Static method
5. Restrictions for the static method
6. Why is the main method static?
7. Static block
8. Can we execute a program without main method?

The **static keyword** in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class than an instance of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block
4. Nested class



1) Java static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable

It makes your program **memory efficient** (i.e., it saves memory).

Understanding the problem without static variable

```
1. class Student{
2.     int rollno;
3.     String name;
4.     String college="ITS";
5. }
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

Java static property is shared to all objects.

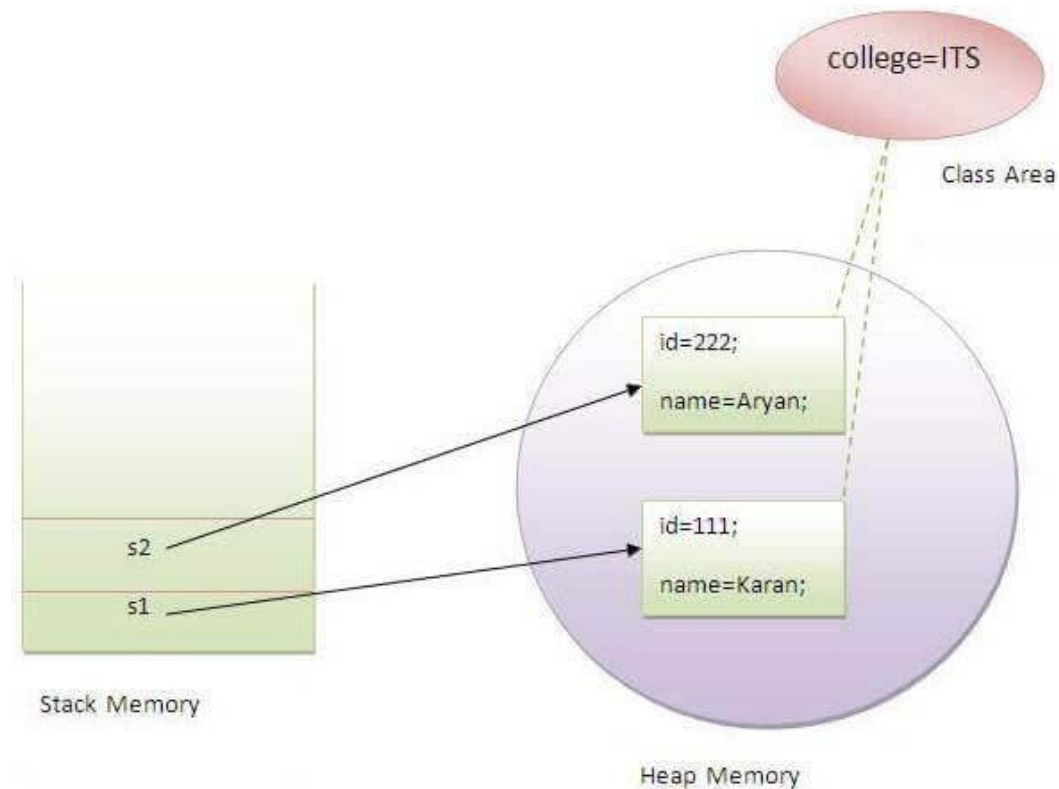
Example of static variable

```
1. //Java Program to demonstrate the use of static variable
```

```
2. class Student{
3.     int rollno;//instance variable
4.     String name;
5.     static String college ="ITS";//static variable
6.     //constructor
7.     Student(int r, String n){
8.         rollno = r;
9.         name = n;
10.    }
11.    //method to display the values
12.    void display (){System.out.println(rollno+" "+name+" "+college);}
13. }
14. //Test class to show the values of objects
15. public class TestStaticVariable1{
16.     public static void main(String args[]){
17.         Student s1 = new Student(111,"Karan");
18.         Student s2 = new Student(222,"Aryan");
19.         //we can change the college of all objects by the single line of code
20.         //Student.college="BBDIT";
21.         s1.display();
22.         s2.display();
23.     }
24. }
```

Output:

```
111 Karan ITS
222 Aryan ITS
```



Program of counter by static variable

As we have mentioned above, static variable will get the memory only once, if any object changes the value of the static variable, it will retain its value.

1. //Java Program to illustrate the use of static variable which
2. //is shared with all objects.
3. **class** Counter2{
4. **static int** count=0;//will get memory only once and retain its value
- 5.
6. Counter2(){
7. count++;//incrementing the value of static variable
8. System.out.println(count);
9. }
- 10.
11. **public static void** main(String args[]){
12. //creating objects
13. Counter2 c1=**new** Counter2();
14. Counter2 c2=**new** Counter2();
15. Counter2 c3=**new** Counter2();
16. }
17. }

Output:

```
1
2
3
```

2) Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Example of static method

```
1. //Java Program to demonstrate the use of a static method.
2. class Student{
3.     int rollno;
4.     String name;
5.     static String college = "ITS";
6.     //static method to change the value of static variable
7.     static void change(){
8.         college = "BBDIT";
9.     }
10.    //constructor to initialize the variable
11.    Student(int r, String n){
12.        rollno = r;
13.        name = n;
14.    }
15.    //method to display values
16.    void display(){System.out.println(rollno+" "+name+" "+college);}
17. }
18. //Test class to create and display the values of object
19. public class TestStaticMethod{
20.     public static void main(String args[]){
21.         Student.change();//calling change method
22.         //creating objects
23.         Student s1 = new Student(111,"Karan");
24.         Student s2 = new Student(222,"Aryan");
25.         Student s3 = new Student(333,"Sonoo");
26.         //calling display method
```

```
27. s1.display();
28. s2.display();
29. s3.display();
30. }
31. }
```

```
Output:111 Karan BBDIT
       222 Aryan BBDIT
       333 Sonoo BBDIT
```

Another example of a static method that performs a normal calculation

```
1. //Java Program to get the cube of a given number using the static method
2.
3. class Calculate{
4.     static int cube(int x){
5.         return x*x*x;
6.     }
7.
8.     public static void main(String args[]){
9.         int result=Calculate.cube(5);
10.    System.out.println(result);
11. }
12. }
```

```
Output:125
```

Restrictions for the static method

There are two main restrictions for the static method. They are:

1. The static method can not use non static data member or call non-static method directly.
2. this and super cannot be used in static context.

```
1. class A{
2.     int a=40;//non static
3.
4.     public static void main(String args[]){
5.         System.out.println(a);
6.     }
7. }
```

```
Output:Compile Time Error
```

Q) Why is the Java main method static?

Ans) It is because the object is not required to call a static method. If it were a non-static method, JVM creates an object first then call main() method that will lead the problem of extra memory allocation.

3) Java static block

- Is used to initialize the static data member.
- It is executed before the main method at the time of classloading.

Example of static block

```
1. class A2{
2.     static{System.out.println("static block is invoked");}
3.     public static void main(String args[]){
4.         System.out.println("Hello main");
5.     }
6. }
```

```
Output:static block is invoked
        Hello main
```

Q) Can we execute a program without main() method?

Ans) No, one of the ways was the static block, but it was possible till JDK 1.6. Since JDK 1.7, it is not possible to execute a Java class without the main method.

```
1. class A3{
2.     static{
3.         System.out.println("static block is invoked");
4.         System.exit(0);
5.     }
6. }
```

Output:

```
static block is invoked
```

Since JDK 1.7 and above, output would be:

```
Error: Main method not found in class A3, please define the main method as:
    public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
```


final

In java, the final is a keyword and it is used with the following things.

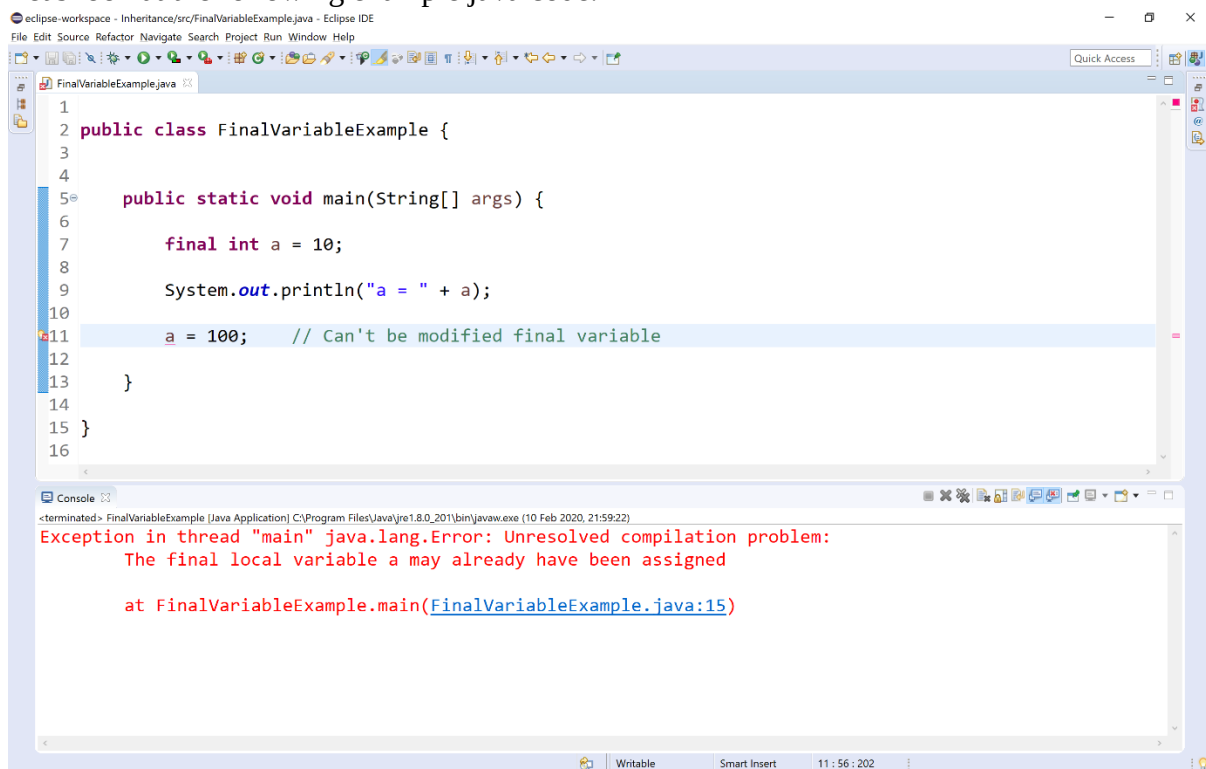
- **With variable (to create constant)**
- **With method (to avoid method overriding)**
- **With class (to avoid inheritance)**

Let's look at each of the above.

final with variables

When a variable defined with the **final** keyword, it becomes a constant, and it does not allow us to modify the value. The variable defined with the final keyword allows only a one-time assignment, once a value assigned to it, never allows us to change it again.

Let's look at the following example java code.



The screenshot shows the Eclipse IDE with a Java file named `FinalVariableExample.java`. The code defines a public class `FinalVariableExample` with a `main` method. Inside the `main` method, a `final int a = 10;` is declared, followed by `System.out.println("a = " + a);`. On line 11, an attempt is made to reassign the value of `a` to 100, which is commented as `// Can't be modified final variable`. The IDE shows a compilation error in the console: `Exception in thread "main" java.lang.Error: Unresolved compilation problem: The final local variable a may already have been assigned at FinalVariableExample.main(FinalVariableExample.java:15)`.

```
1 public class FinalVariableExample {
2
3
4
5     public static void main(String[] args) {
6
7         final int a = 10;
8
9         System.out.println("a = " + a);
10
11        a = 100;    // Can't be modified final variable
12    }
13
14
15 }
16
```

Console Output:

```
<terminated> FinalVariableExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (10 Feb 2020, 21:59:22)
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    The final local variable a may already have been assigned

    at FinalVariableExample.main(FinalVariableExample.java:15)
```

final with methods

When a method defined with the **final** keyword, it does not allow it to override. The final method extends to the child class, but the child class can not override or re-define it. It must be used as it has implemented in the parent class.

Let's look at the following example java code.

```
1 class ParentClass{
2
3     int num = 10;
4
5     final void showData() {
6         System.out.println("Inside ParentClass showData() method");
7         System.out.println("num = " + num);
8     }
9
10 }
11
12 class ChildClass extends ParentClass{
13
14     void showData() {
15         System.out.println("Inside ChildClass showData() method");
16         System.out.println("num = " + num);
17     }
18 }
19
20 public class FinalKeywordExample {
21
22     public static void main(String[] args) {
```

Console

<terminated> SuperKeywordExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (10 Feb 2020, 22:47:32)

Exception in thread "main" java.lang.VerifyError: class ChildClass overrides final method showData().()V
at java.lang.ClassLoader.defineClass1(Native Method)
at java.lang.ClassLoader.defineClass(Unknown Source)
at java.security.SecureClassLoader.defineClass(Unknown Source)

Cannot override the final method from ParentClass

final with class

When a class defined with final keyword, it can not be extended by any other class.

Let's look at the following example java code.

```
1 final class ParentClass{
2
3     int num = 10;
4
5     void showData() {
6         System.out.println("Inside ParentClass showData() method");
7         System.out.println("num = " + num);
8     }
9
10 }
11
12 class ChildClass extends ParentClass{
13
14 }
15
16 public class FinalKeywordExample {
17
18     public static void main(String[] args) {
19
20         ChildClass obj = new ChildClass();
21
22     }
```

Console

<terminated> FinalKeywordExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (10 Feb 2020, 23:50:23)

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
The type ChildClass cannot subclass the final class ParentClass
at ChildClass.<init>(FinalKeywordExample.java:12)

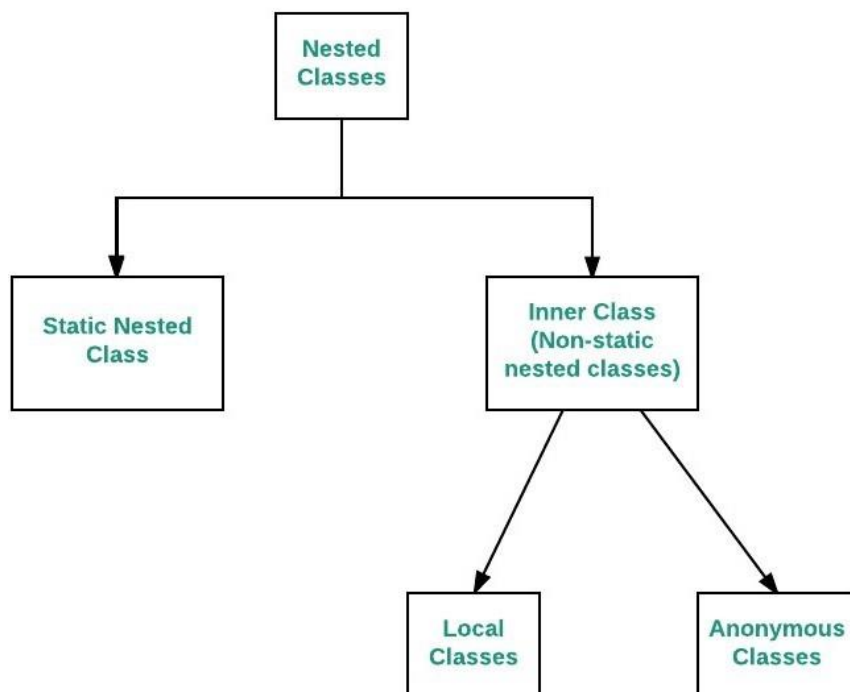
nested and inner classes

In Java, it is possible to define a class within another class, such classes are known as *nested* classes. They enable you to logically group classes that are only used in one place, thus this increases the use of encapsulation, and creates more readable and maintainable code.

- The scope of a nested class is bounded by the scope of its enclosing class. Thus in below example, class *NestedClass* does not exist independently of class *OuterClass*.
- A nested class has access to the members, including private members, of the class in which it is nested. The reverse is also true i.e., the enclosing class can access the members of the nested class.
- A nested class is also a member of its enclosing class.
- As a member of its enclosing class, a nested class can be declared *private*, *public*, *protected*, or *package private*(default).
- Nested classes are divided into two categories:
 1. **static nested class** : Nested classes that are declared *static* are called static nested classes.
 2. **inner class** : An inner class is a non-static nested class.

Syntax:

```
class OuterClass
{
...
    class NestedClass
    {
        ...
    }
}
```



Static nested classes

In the case of normal or regular inner classes, without an outer class object existing, there cannot be an inner class object. i.e., an object of the inner class is always strongly associated with an outer class object. But in the case of static nested class, Without an outer class object existing, there may be a static nested class object. i.e., an object of a static nested class is not strongly associated with the outer class object.

As with class methods and variables, a static nested class is associated with its outer class. And like static class methods, a static nested class cannot refer directly to instance variables or methods defined in its enclosing class: it can use them only through an object reference. They are accessed using the enclosing class name.

OuterClass.StaticNestedClass

For example, to create an object for the static nested class, use this syntax:

```
OuterClass.StaticNestedClass nestedObject =  
    new OuterClass.StaticNestedClass();
```

```
// Java program to demonstrate accessing  
// a static nested class
```

```
// outer class  
class OuterClass  
{  
    // static member  
    static int outer_x = 10;  
  
    // instance(non-static) member  
    int outer_y = 20;  
  
    // private member  
    private static int outer_private = 30;  
  
    // static nested class  
    static class StaticNestedClass  
    {  
        void display()  
        {  
            // can access static member of outer class  
            System.out.println("outer_x = " + outer_x);  
  
            // can access display private static member of outer class  
            System.out.println("outer_private = " + outer_private);  
  
            // The following statement will give compilation error  
            // as static nested class cannot directly access non-static members  
            // System.out.println("outer_y = " + outer_y);  
        }  
    }  
}
```

```
}
```

```
// Driver class
public class StaticNestedClassDemo
{
    public static void main(String[] args)
    {
        // accessing a static nested class
        OuterClass.StaticNestedClass nestedObject = new OuterClass.StaticNestedClass();

        nestedObject.display();
    }
}
```

Output:

```
outer_x = 10
outer_private = 30
```

Inner classes

To instantiate an inner class, you must first instantiate the outer class. Then, create the inner object within the outer object with this syntax:

```
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

There are two special kinds of inner classes :

1. Local inner classes
2. Anonymous inner classes

```
// Java program to demonstrate accessing
// a inner class
```

```
// outer class
class OuterClass
{
    // static member
    static int outer_x = 10;

    // instance(non-static) member
    int outer_y = 20;

    // private member
    private int outer_private = 30;

    // inner class
```

```

class InnerClass
{
    void display()
    {
        // can access static member of outer class
        System.out.println("outer_x = " + outer_x);

        // can also access non-static member of outer class
        System.out.println("outer_y = " + outer_y);

        // can also access a private member of the outer class
        System.out.println("outer_private = " + outer_private);
    }
}

// Driver class
public class InnerClassDemo
{
    public static void main(String[] args)
    {
        // accessing an inner class
        OuterClass outerObject = new OuterClass();
        OuterClass.InnerClass innerObject = outerObject.new InnerClass();

        innerObject.display();
    }
}

```

Output:

```

outer_x = 10
outer_y = 20
outer_private = 30

```

Comparison between normal or regular class and static nested class

S.NO	Normal/Regular inner class	Static nested class
1.	Without an outer class object existing, there cannot be an inner class object. That is, the inner class object is always associated with the outer class object.	Without an outer class object existing, there may be a static nested class object. That is, static nested class object is not associated with the outer class object.

S.NO	Normal/Regular inner class	Static nested class
2.	Inside normal/regular inner class, static members can't be declared.	Inside static nested class, static members can be declared.
3.	As main() method can't be declared, regular inner class can't be invoked directly from the command prompt.	As main() method can be declared, the static nested class can be invoked directly from the command prompt.
4.	Both static and non static members of outer class can be accessed directly.	Only a static member of outer class can be accessed directly.

exploring string class

A string is a sequence of characters surrounded by double quotations. In a java programming language, a string is the object of a built-in class **String**.

In the background, the string values are organized as an array of a character data type.

The string created using a character array can not be extended. It does not allow to append more characters after its definition, but it can be modified.

Let's look at the following example java code.

Example

```
char[] name = {'J', 'a', 'v', 'a', ' ', 'T', 'u', 't', 'o', 'r', 'i', 'a', 'l', 's'};
//name[14] = '@'; //ArrayIndexOutOfBoundsException
name[5] = '-';
System.out.println(name);
```

The **String** class defined in the package **java.lang** package. The **String** class implements **Serializable**, **Comparable**, and **CharSequence** interfaces.

The string created using the **String** class can be extended. It allows us to add more characters after its definition, and also it can be modified.

Let's look at the following example java code.

Example

```
String siteName = "btechsmartclass.com";
siteName = "www.btechsmartclass.com";
```

Creating String object in java

In java, we can use the following two ways to create a string object.

- Using string literal
- Using String constructor

Let's look at the following example java code.

Example

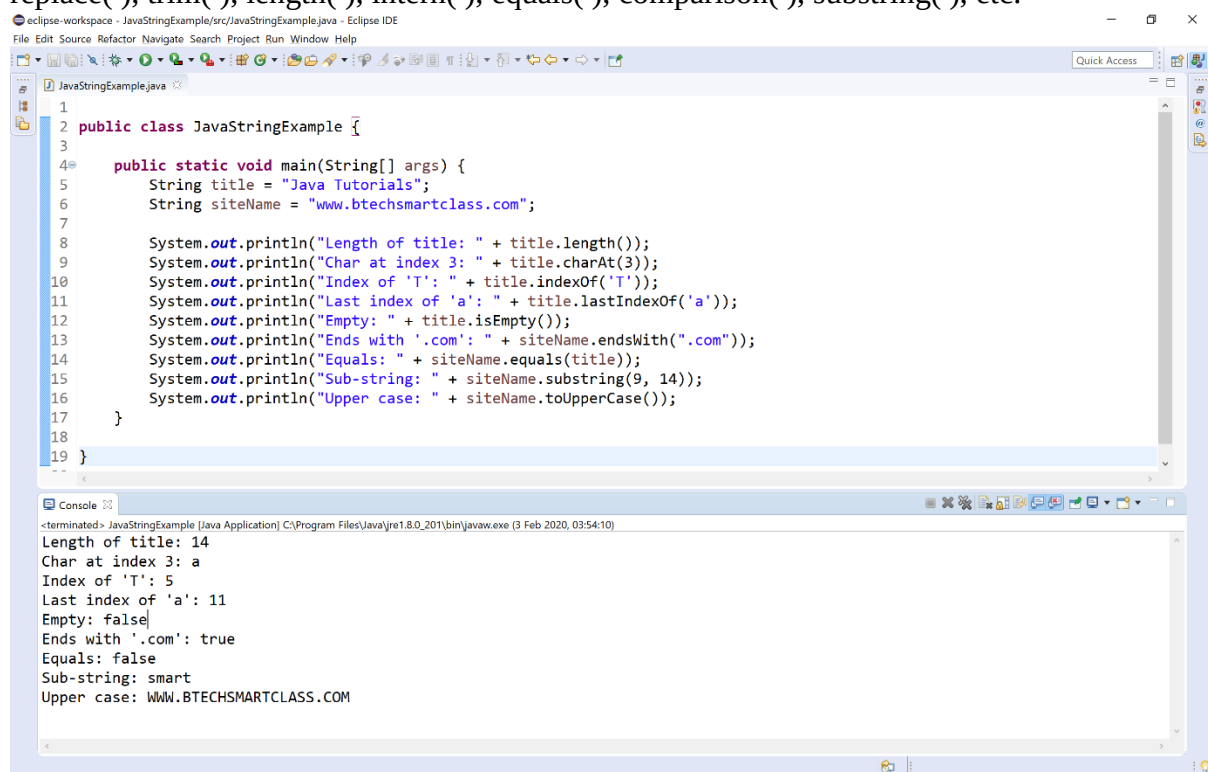
```
String title = "Java Tutorials";           // Using literals
```

```
String siteName = new String("www.btechsmartclass.com");    // Using constructor
```

 The String class constructor accepts both string and character array as an argument.

String handling methods

In java programming language, the String class contains various methods that can be used to handle string data values. It contains methods like concat(), compareTo(), split(), join(), replace(), trim(), length(), intern(), equals(), comparison(), substring(), etc.



The screenshot shows the Eclipse IDE with a Java file named `JavaStringExample.java`. The code defines a public class `JavaStringExample` with a `main` method that takes an array of strings as an argument. Inside the `main` method, two strings are created: `title = "Java Tutorials"` and `siteName = "www.btechsmartclass.com"`. The code then uses various `String` methods to demonstrate their functionality, including `length()`, `charAt()`, `indexOf()`, `lastIndexOf()`, `isEmpty()`, `endsWith()`, `equals()`, `substring()`, and `toUpperCase()`. The output of the program is displayed in the console window at the bottom, showing the results of each method call.

```
1 public class JavaStringExample {
2
3
4     public static void main(String[] args) {
5         String title = "Java Tutorials";
6         String siteName = "www.btechsmartclass.com";
7
8         System.out.println("Length of title: " + title.length());
9         System.out.println("Char at index 3: " + title.charAt(3));
10        System.out.println("Index of 'T': " + title.indexOf('T'));
11        System.out.println("Last index of 'a': " + title.lastIndexOf('a'));
12        System.out.println("Empty: " + title.isEmpty());
13        System.out.println("Ends with '.com': " + siteName.endsWith(".com"));
14        System.out.println("Equals: " + siteName.equals(title));
15        System.out.println("Sub-string: " + siteName.substring(9, 14));
16        System.out.println("Upper case: " + siteName.toUpperCase());
17    }
18 }
19 }
```

```
<terminated> JavaStringExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (3 Feb 2020, 03:54:10)
Length of title: 14
Char at index 3: a
Index of 'T': 5
Last index of 'a': 11
Empty: false
Ends with '.com': true
Equals: false
Sub-string: smart
Upper case: WWW.BTECHSMARTCLASS.COM
```

using command-line arguments

A command-line argument is an information that directly follows the program's name on the command line when it is executed. To access the command-line arguments inside a Java program is quite easy. They are stored as strings in the String array passed to `main()`.

Example

The following program displays all of the command-line arguments that it is called with -


```
public class CommandLine {  
    public static void main(String args[]) {  
        for(int i = 0; i<args.length; i++) {  
            System.out.println("args[" + i + "]: " + args[i]);  
        }  
    }  
}
```

Try executing this program as shown here -

\$java CommandLine this is a command line 200 -100

Output

This will produce the following result -

args[0]: this

args[1]: is

args[2]: a

args[3]: command

args[4]: line

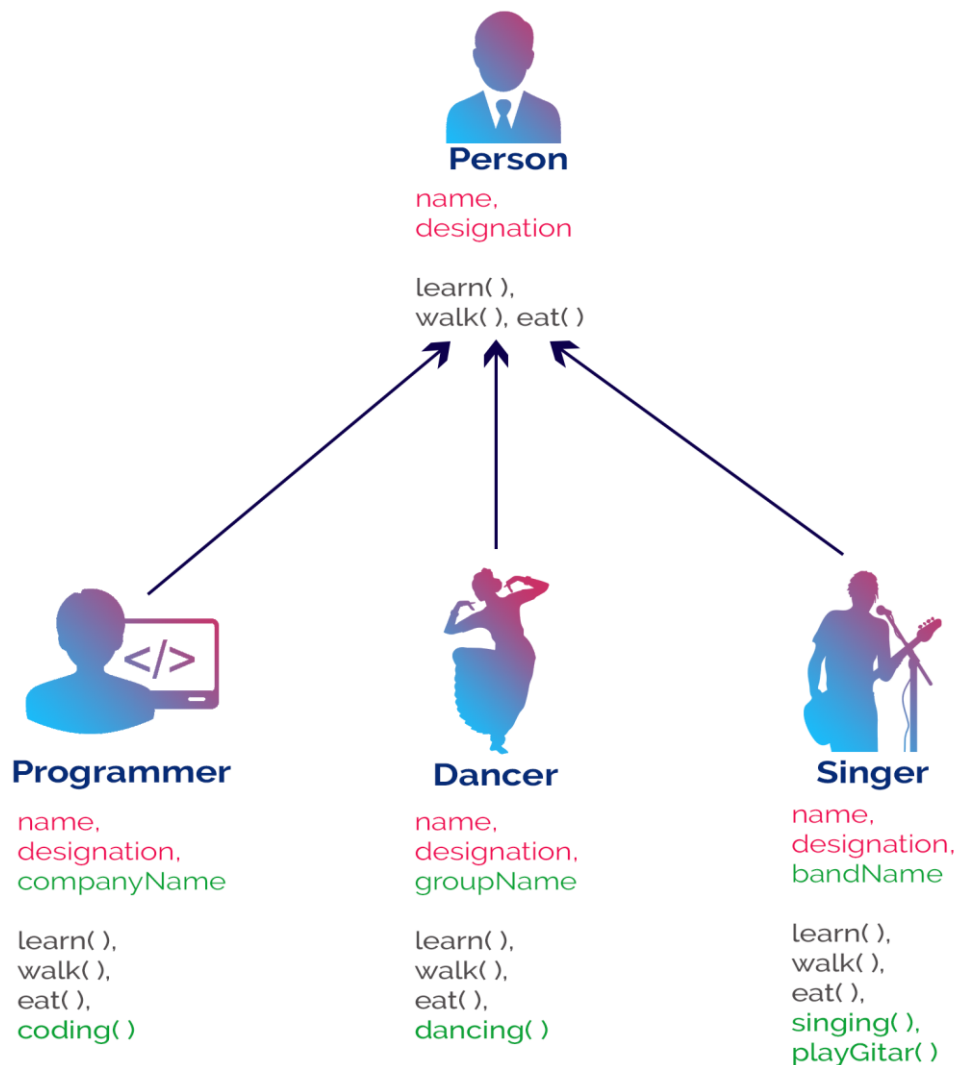
args[5]: 200

args[6]: -100

Inheritance: Inheritance concept

Inheritance Concept

The inheritance is a very useful and powerful concept of object-oriented programming. In java, using the inheritance concept, we can use the existing features of one class in another class. The inheritance provides a greater advantage called code re-usability. With the help of code re-usability, the commonly used code in an application need not be written again and again.



www.btechsmartclass.com

The inheritance can be defined as follows.

The inheritance is the process of acquiring the properties of one class to another class.

Inheritance Basics

In inheritance, we use the terms like parent class, child class, base class, derived class, superclass, and subclass.

The **Parent class** is the class which provides features to another class. The parent class is also known as **Base class** or **Superclass**.

The **Child class** is the class which receives features from another class. The child class is also known as the **Derived Class** or **Subclass**.

In the inheritance, the child class acquires the features from its parent class. But the parent class never acquires the features from its child class.

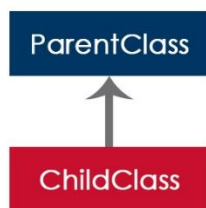
types of inheritance

There are five types of inheritances, and they are as follows.

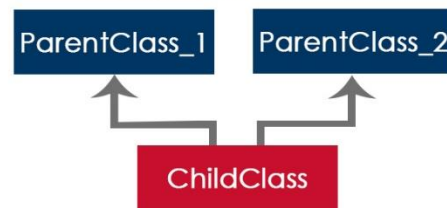
- **Simple Inheritance (or) Single Inheritance**
- **Multiple Inheritance**
- **Multi-Level Inheritance**
- **Hierarchical Inheritance**
- **Hybrid Inheritance**

The following picture illustrates how various inheritances are implemented.

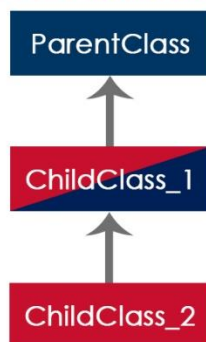
Simple Inheritance



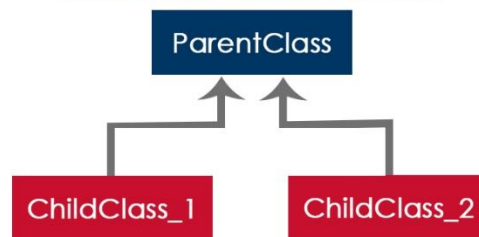
Multiple Inheritance



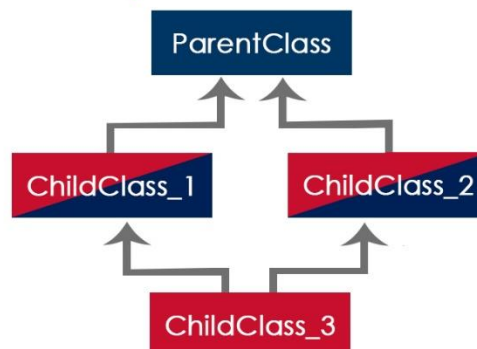
Multi Level Inheritance



Hierarchical Inheritance



Hybrid Inheritance



The java programming language does not support multiple inheritance type. However, it provides an alternate with the concept of interfaces.

Creating Child Class in java

In java, we use the keyword **extends** to create a child class. The following syntax used to create a child class in java.

Syntax

```
class <ChildClassName> extends <ParentClassName>{  
    ...  
    //Implementation of child class  
    ...  
}
```

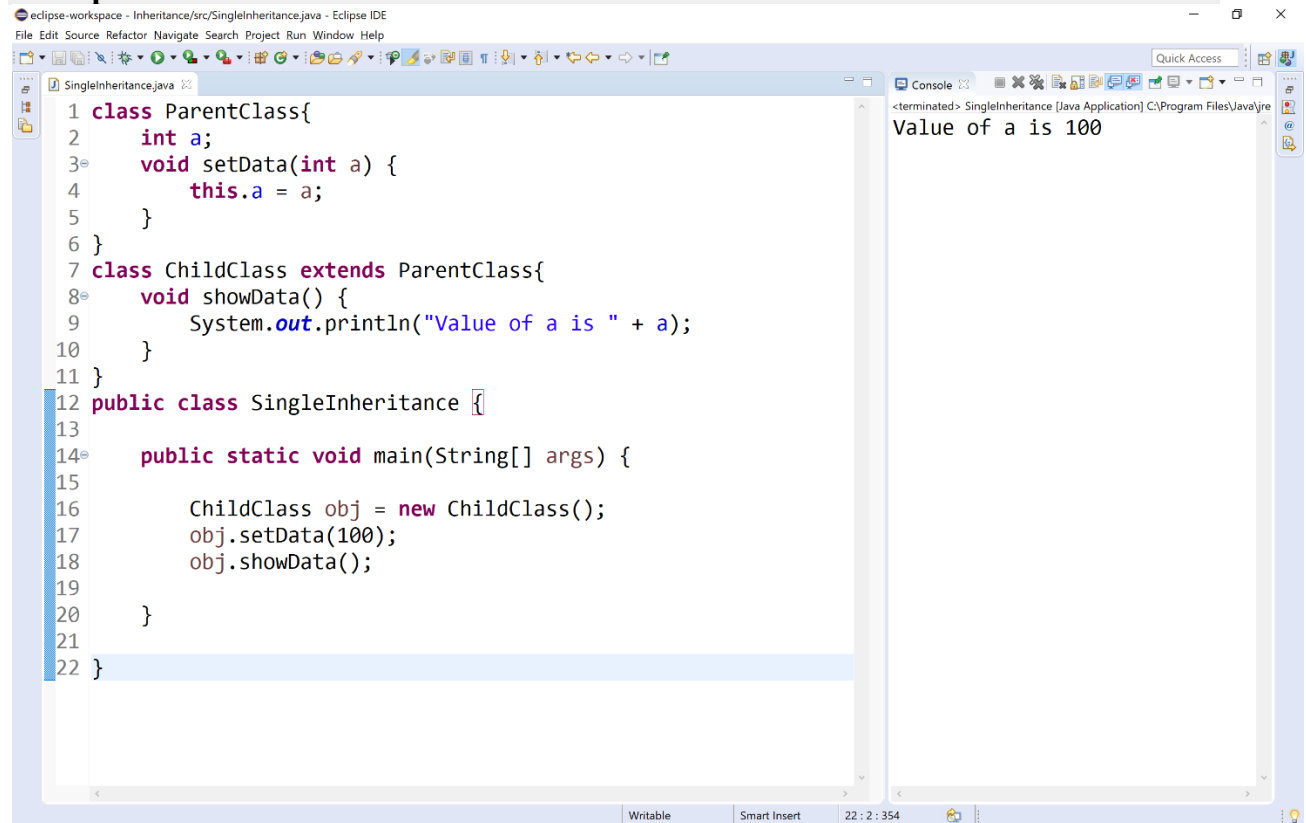
In a java programming language, a class extends only one class. Extending multiple classes is not allowed in java.

Let's look at individual inheritance types and how they get implemented in java with an example.

Single Inheritance in java

In this type of inheritance, one child class derives from one parent class. Look at the following example code.

Example



The screenshot shows the Eclipse IDE with a Java project named 'Inheritance'. The source code in 'SingleInheritance.java' defines a 'ParentClass' with an 'int a' and a 'setData' method. 'ChildClass' extends 'ParentClass' and overrides 'showData' to print the value of 'a'. The 'SingleInheritance' class has a 'main' method that creates a 'ChildClass' object, sets 'a' to 100, and calls 'showData'. The console output shows 'Value of a is 100'.

```
1 class ParentClass{  
2     int a;  
3     void setData(int a) {  
4         this.a = a;  
5     }  
6 }  
7 class ChildClass extends ParentClass{  
8     void showData() {  
9         System.out.println("Value of a is " + a);  
10    }  
11 }  
12 public class SingleInheritance {  
13  
14     public static void main(String[] args) {  
15  
16         ChildClass obj = new ChildClass();  
17         obj.setData(100);  
18         obj.showData();  
19     }  
20 }  
21  
22 }
```

Console Output:
<terminated> SingleInheritance [Java Application] C:\Program Files\Java\jre...
Value of a is 100

Multi-level Inheritance in java

In this type of inheritance, the child class derives from a class which already derived from another class. Look at the following example java code.

Example

```
class ParentClass{
    int a;
    void setData(int a) {
        this.a = a;
    }
}
class ChildClass extends ParentClass{
    void showData() {
        System.out.println("Value of a is " + a);
    }
}
class ChildChildClass extends ChildClass{
    void display() {
        System.out.println("Inside ChildChildClass!");
    }
}
public class MultipleInheritance {

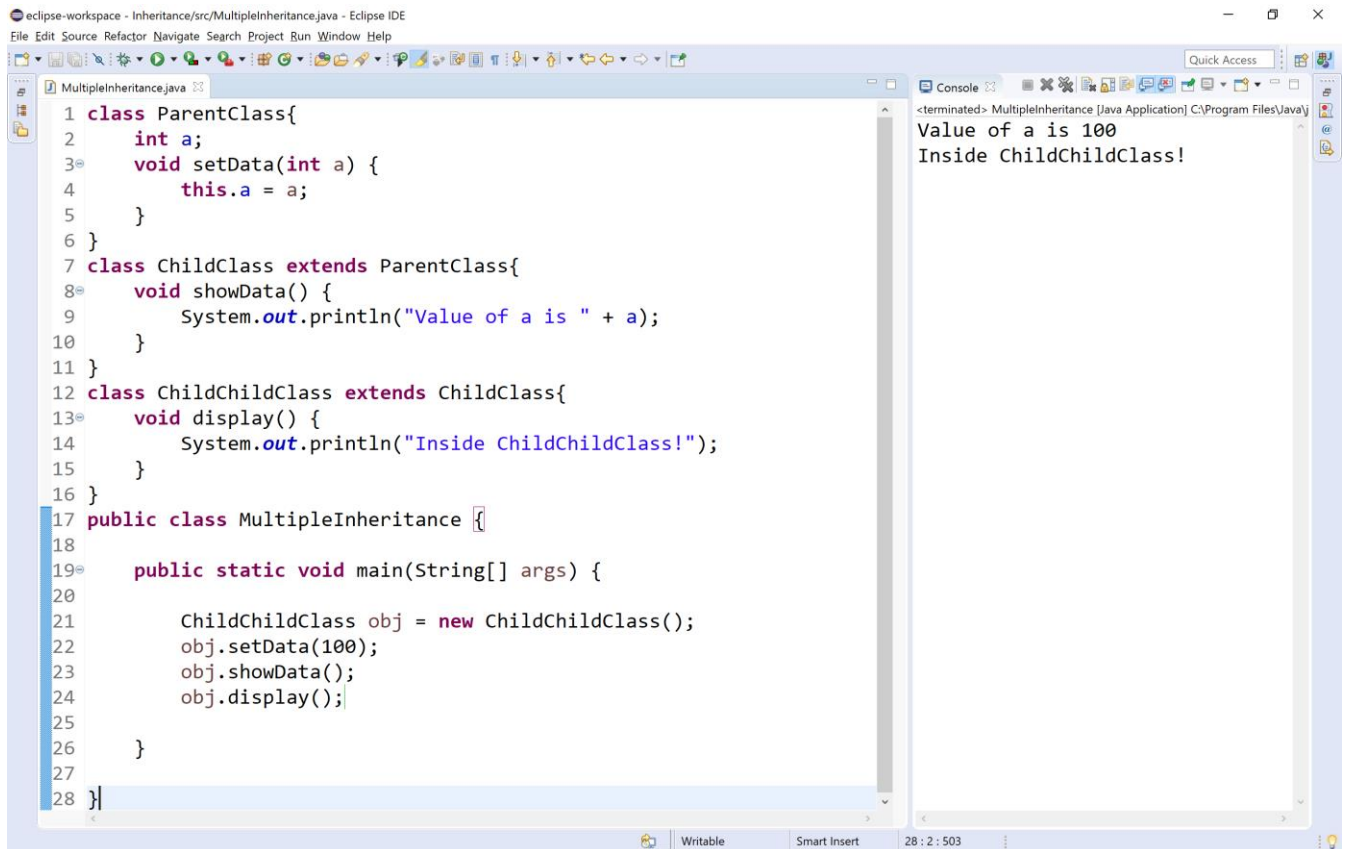
    public static void main(String[] args) {

        ChildChildClass obj = new ChildChildClass();
        obj.setData(100);
        obj.showData();
        obj.display();

    }

}
```

When we run this code, it produce the following output.



Hierarchical Inheritance in java

In this type of inheritance, two or more child classes derive from one parent class. Look at the following example java code.

Example

```
class ParentClass{
    int a;
    void setData(int a) {
        this.a = a;
    }
}
class ChildClass extends ParentClass{
    void showData() {
        System.out.println("Inside ChildClass!");
        System.out.println("Value of a is " + a);
    }
}
class ChildClassToo extends ParentClass{
    void display() {
        System.out.println("Inside ChildClassToo!");
        System.out.println("Value of a is " + a);
    }
}
public class HierarchicalInheritance {
```

```

        public static void main(String[] args) {

            ChildClass child_obj = new ChildClass();
            child_obj.setData(100);
            child_obj.showData();

            ChildClassToo childToo_obj = new ChildClassToo();
            childToo_obj.setData(200);
            childToo_obj.display();

        }
    }
}

```

When we run this code, it produce the following output.

The screenshot shows the Eclipse IDE interface. The main editor window displays the Java code for 'HierarchicalInheritance.java'. The code defines a 'ParentClass' with an 'int a' attribute and a 'setData' method. It then defines two subclasses: 'ChildClass' which extends 'ParentClass' and overrides 'showData' to print 'Inside ChildClass!' and 'Value of a is ' followed by the value of 'a'; and 'ChildClassToo' which also extends 'ParentClass' and overrides 'display' to print 'Inside ChildClassToo!' and 'Value of a is ' followed by the value of 'a'. The 'main' method in the 'HierarchicalInheritance' class creates instances of both subclasses and calls their respective methods.

The console window on the right shows the output of the program, which is terminated. The output consists of three lines: 'Inside ChildClass!', 'Value of a is 100', and 'Inside ChildClassToo!', 'Value of a is 200'.

```

1 class ParentClass{
2     int a;
3     void setData(int a) {
4         this.a = a;
5     }
6 }
7 class ChildClass extends ParentClass{
8     void showData() {
9         System.out.println("Inside ChildClass!");
10        System.out.println("Value of a is " + a);
11    }
12 }
13 class ChildClassToo extends ParentClass{
14     void display() {
15         System.out.println("Inside ChildClassToo!");
16         System.out.println("Value of a is " + a);
17    }
18 }
19 public class HierarchicalInheritance {
20
21     public static void main(String[] args) {
22
23         ChildClass child_obj = new ChildClass();
24         child_obj.setData(100);
25         child_obj.showData();
26
27         ChildClassToo childToo_obj = new ChildClassToo();
28         childToo_obj.setData(200);
29         childToo_obj.display();
30
31    }
32 }
33 }

```

```

<terminated> MultipleInheritance [Java Application] C:\Program Files\Java\j
Inside ChildClass!
Value of a is 100
Inside ChildClassToo!
Value of a is 200

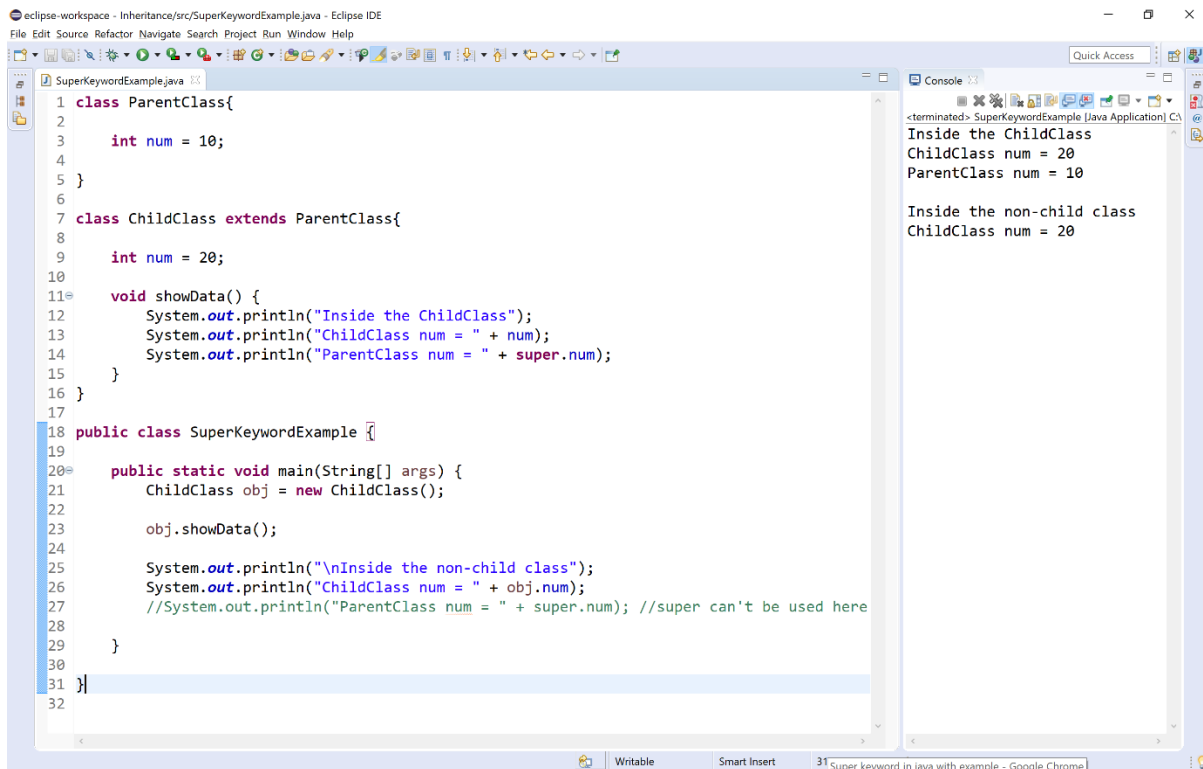
```

Hybrid Inheritance in java

super to refer parent class data members

When both parent class and child class have data members with the same name, then the super keyword is used to refer to the parent class data member from child class.

Let's look at the following example java code.



```
1 class ParentClass{
2
3     int num = 10;
4
5 }
6
7 class ChildClass extends ParentClass{
8
9     int num = 20;
10
11     void showData() {
12         System.out.println("Inside the ChildClass");
13         System.out.println("ChildClass num = " + num);
14         System.out.println("ParentClass num = " + super.num);
15     }
16 }
17
18 public class SuperKeywordExample {
19
20     public static void main(String[] args) {
21         ChildClass obj = new ChildClass();
22
23         obj.showData();
24
25         System.out.println("\nInside the non-child class");
26         System.out.println("ChildClass num = " + obj.num);
27         //System.out.println("ParentClass num = " + super.num); //super can't be used here
28     }
29 }
30
31
32 }
```

Console Output:

```
<terminated> SuperKeywordExample [Java Application] C:\
Inside the ChildClass
ChildClass num = 20
ParentClass num = 10

Inside the non-child class
ChildClass num = 20
```

super to refer parent class method

When both parent class and child class have method with the same name, then the super keyword is used to refer to the parent class method from child class.

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java project named 'Inheritance/src/SuperKeywordExample.java'. The code defines a `ParentClass` with a `showData()` method that prints 'Inside the ParentClass showData method' and 'ChildClass num = ' + num1. A `ChildClass` extends `ParentClass` and overrides `showData()` to print 'Inside the ChildClass showData method' and 'ChildClass num = ' + num2, followed by `super.showData();`. A `SuperKeywordExample` class has a `main` method that creates a `ChildClass` object and calls `obj.showData();`. The console output shows the execution results: 'Inside the ChildClass showData method' and 'ChildClass num = 20' followed by 'Inside the ParentClass showData method' and 'ChildClass num = 10'.

```
1 class ParentClass{
2
3     int num1 = 10;
4
5     void showData() {
6         System.out.println("\nInside the ParentClass showData method");
7         System.out.println("ChildClass num = " + num1);
8     }
9 }
10
11 class ChildClass extends ParentClass{
12
13     int num2 = 20;
14
15     void showData() {
16         System.out.println("\nInside the ChildClass showData method");
17         System.out.println("ChildClass num = " + num2);
18
19         super.showData();
20     }
21 }
22
23
24 public class SuperKeywordExample {
25
26     public static void main(String[] args) {
27         ChildClass obj = new ChildClass();
28
29         obj.showData();
30         //super.showData(); // super can't be used here
31     }
32 }
33 }
```

Console Output:

```
<terminated> SuperKeywordExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin
Inside the ChildClass showData method
ChildClass num = 20

Inside the ParentClass showData method
ChildClass num = 10
```

super to call parent class constructor

When an object of child class is created, it automatically calls the parent class default-constructor before it's own. But, the parameterized constructor of parent class must be called explicitly using the **super** keyword inside the child class constructor.

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java project named 'Inheritance/src/SuperKeywordExample.java'. The code defines a `ParentClass` with two constructors: a default constructor that prints 'Inside the ParentClass default constructor' and a parameterized constructor that takes an `int` value and prints 'Inside the ParentClass parameterized constructor'. A `ChildClass` extends `ParentClass` and overrides the default constructor to call `super(100);` first, then prints 'Inside the ChildClass constructor' and sets `num2 = 200`. A `SuperKeywordExample` class has a `main` method that creates a `ChildClass` object. The console output shows the execution results: 'Inside the ParentClass parameterized constructor' followed by 'Inside the ChildClass constructor'.

```
1 class ParentClass{
2
3     int num1;
4
5     ParentClass(){
6         System.out.println("\nInside the ParentClass default constructor");
7         num1 = 10;
8     }
9
10    ParentClass(int value){
11        System.out.println("\nInside the ParentClass parameterized constructor");
12        num1 = value;
13    }
14 }
15
16 class ChildClass extends ParentClass{
17
18     int num2;
19
20    ChildClass(){
21        super(100);
22        System.out.println("\nInside the ChildClass constructor");
23        num2 = 200;
24    }
25 }
26
27 public class SuperKeywordExample {
28
29     public static void main(String[] args) {
30
31         ChildClass obj = new ChildClass();
32     }
33 }
34 }
```

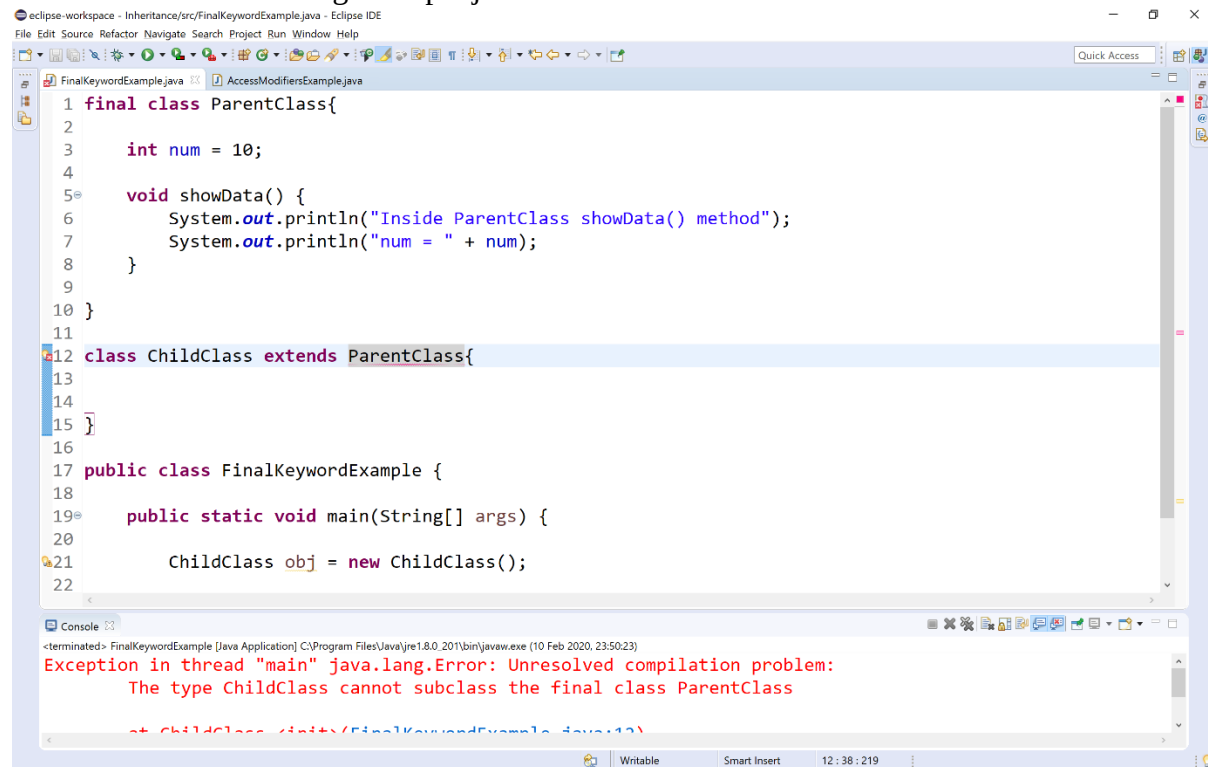
Console Output:

```
<terminated> SuperKeywordExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin
Inside the ParentClass parameterized constructor
Inside the ChildClass constructor
```

To call the parameterized constructor of the parent class, the super keyword must be the first statement inside the child class constructor, and we must pass the parameter values.

final with class

When a class defined with final keyword, it can not be extended by any other class. Let's look at the following example java code.



```
1 final class ParentClass{
2
3     int num = 10;
4
5     void showData() {
6         System.out.println("Inside ParentClass showData() method");
7         System.out.println("num = " + num);
8     }
9
10 }
11
12 class ChildClass extends ParentClass{
13
14
15 }
16
17 public class FinalKeywordExample {
18
19     public static void main(String[] args) {
20
21         ChildClass obj = new ChildClass();
22     }
23 }
```

Console Output:

```
<terminated> FinalKeywordExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (10 Feb 2020, 23:50:23)
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    The type ChildClass cannot subclass the final class ParentClass
    at ChildClass.<init>(FinalKeywordExample.java:12)
```

Polymorphism - dynamic binding

The polymorphism is the process of defining same method with different implementation. That means creating multiple methods with different behaviors.

In java, polymorphism implemented using method overloading and method overriding.

Ad hoc polymorphism

The ad hoc polymorphism is a technique used to define the same method with different implementations and different arguments. In a java programming language, ad hoc polymorphism carried out with a method overloading concept.

In ad hoc polymorphism the method binding happens at the time of compilation. Ad hoc polymorphism is also known as compile-time polymorphism. Every function call binded with the respective overloaded method based on the arguments.

The ad hoc polymorphism implemented within the class only.

Let's look at the following example java code.

```
1 import java.util.Arrays;
2
3 public class AdHocPolymorphismExample {
4
5     void sorting(int[] list) {
6         Arrays.parallelSort(list);
7         System.out.println("Integers after sort: " + Arrays.toString(list) );
8     }
9     void sorting(String[] names) {
10        Arrays.parallelSort(names);
11        System.out.println("Names after sort: " + Arrays.toString(names) );
12    }
13
14    public static void main(String[] args) {
15
16        AdHocPolymorphismExample obj = new AdHocPolymorphismExample();
17        int list[] = {2, 3, 1, 5, 4};
18        obj.sorting(list); // Calling with integer array
19
20        String[] names = {"rama", "raja", "shyam", "seeta"};
21        obj.sorting(names); // Calling with String array
22    }
23 }
24
```

Console

```
<terminated> AdHocPolymorphismExample [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (11 Feb 2020, 15:21:40)
Integers after sort: [1, 2, 3, 4, 5]
Names after sort: [raja, rama, seeta, shyam]
```

Pure polymorphism

The pure polymorphism is a technique used to define the same method with the same arguments but different implementations. In a java programming language, pure polymorphism carried out with a method overriding concept.

In pure polymorphism, the method binding happens at run time. Pure polymorphism is also known as run-time polymorphism. Every function call binding with the respective overridden method based on the object reference.

When a child class has a definition for a member function of the parent class, the parent class function is said to be overridden.

The pure polymorphism implemented in the inheritance concept only.

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java project named 'PurePolymorphism'. The main editor displays the following code:

```
1 class ParentClass{
2
3     int num = 10;
4
5     void showData() {
6         System.out.println("Inside ParentClass showData() method");
7         System.out.println("num = " + num);
8     }
9
10 }
11
12 class ChildClass extends ParentClass{
13
14     void showData() {
15         System.out.println("Inside ChildClass showData() method");
16         System.out.println("num = " + num);
17     }
18 }
19
20 public class PurePolymorphism {
21
22     public static void main(String[] args) {
23
24         ParentClass obj = new ParentClass();
25         obj.showData();
26
27         obj = new ChildClass();
28         obj.showData();
29
30     }
31 }
32
```

The console on the right shows the output of the program:

```
<terminated> PurePolymorphism [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\jav
Inside ParentClass showData() method
num = 10
Inside ChildClass showData() method
num = 10
```

method overriding

The method overriding is the process of re-defining a method in a child class that is already defined in the parent class. When both parent and child classes have the same method, then that method is said to be the overriding method.

The method overriding enables the child class to change the implementation of the method which acquired from parent class according to its requirement.

In the case of the method overriding, the method binding happens at run time. The method binding which happens at run time is known as late binding. So, the method overriding follows late binding.

The method overriding is also known as **dynamic method dispatch** or **run time polymorphism** or **pure polymorphism**.

Let's look at the following example java code.

The screenshot shows the Eclipse IDE with a Java project named 'PurePolymorphism'. The main editor displays the source code for 'PurePolymorphism.java'. It defines a 'ParentClass' with a 'num' attribute (10) and a 'showData()' method. A 'ChildClass' extends 'ParentClass' and overrides 'showData()'. A 'PurePolymorphism' class contains a 'main' method that creates instances of both 'ParentClass' and 'ChildClass' and calls 'showData()' on each. The console on the right shows the output of the program, confirming that the 'ChildClass' method is called for both objects, displaying 'num = 10'.

```
1 class ParentClass{
2
3     int num = 10;
4
5     void showData() {
6         System.out.println("Inside ParentClass showData() method");
7         System.out.println("num = " + num);
8     }
9
10 }
11
12 class ChildClass extends ParentClass{
13
14     void showData() {
15         System.out.println("Inside ChildClass showData() method");
16         System.out.println("num = " + num);
17     }
18 }
19
20 public class PurePolymorphism {
21
22     public static void main(String[] args) {
23
24         ParentClass obj = new ParentClass();
25         obj.showData();
26
27         obj = new ChildClass();
28         obj.showData();
29
30     }
31 }
32
```

Console Output:

```
<terminated> PurePolymorphism [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\jav
Inside ParentClass showData() method
num = 10
Inside ChildClass showData() method
num = 10
```

10 Rules for method overriding

While overriding a method, we must follow the below list of rules.

- Static methods can not be overridden.
- Final methods can not be overridden.
- Private methods can not be overridden.
- Constructor can not be overridden.
- An abstract method must be overridden.
- Use super keyword to invoke overridden method from child class.
- The return type of the overriding method must be same as the parent has it.
- The access specifier of the overriding method can be changed, but the visibility must increase but not decrease. For example, a protected method in the parent class can be made public, but not private, in the child class.
- If the overridden method does not throw an exception in the parent class, then the child class overriding method can only throw the unchecked exception, throwing a checked exception is not allowed.
- If the parent class overridden method does throw an exception, then the child class overriding method can only throw the same, or subclass exception, or it may not throw any exception.

abstract classes and methods

An abstract class is a class that created using abstract keyword. In other words, a class prefixed with abstract keyword is known as an abstract class.

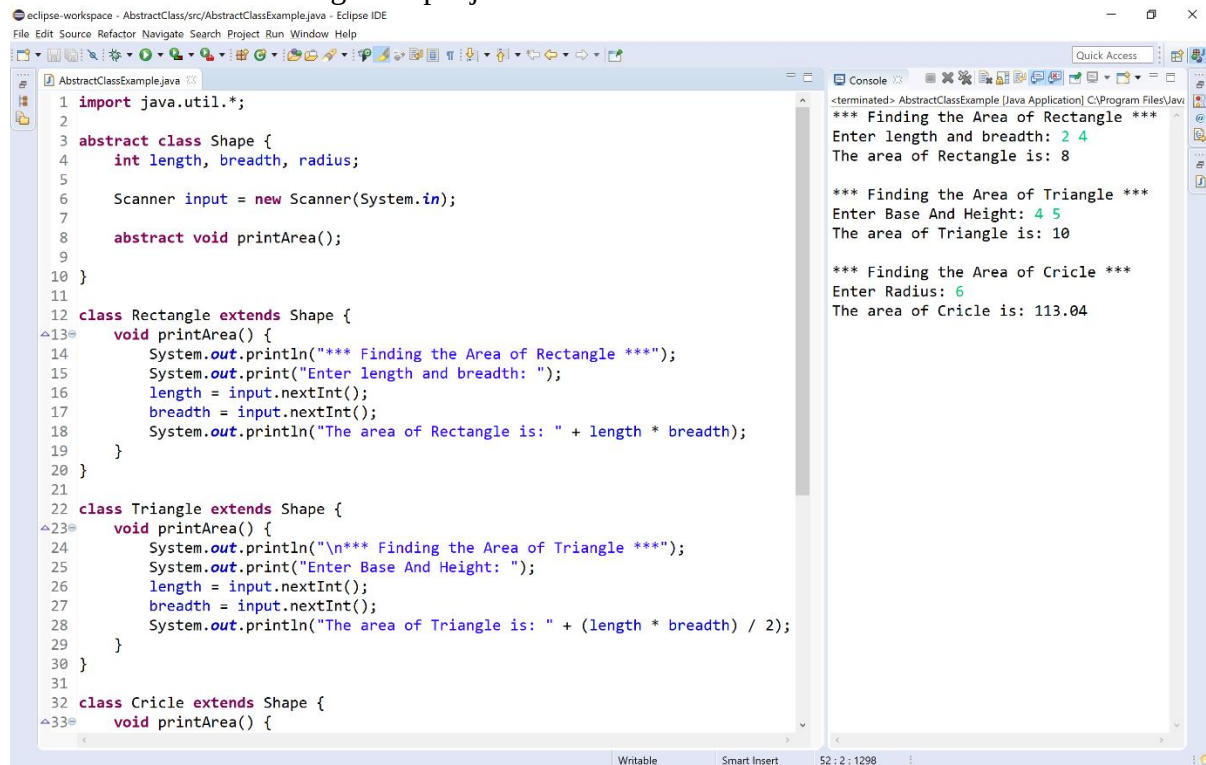
In java, an abstract class may contain abstract methods (methods without implementation) and also non-abstract methods (methods with implementation).

We use the following syntax to create an abstract class.

Syntax

```
abstract class <ClassName>{  
  
    ...  
  
}
```

Let's look at the following example java code.



```
1 import java.util.*;  
2  
3 abstract class Shape {  
4     int length, breadth, radius;  
5  
6     Scanner input = new Scanner(System.in);  
7  
8     abstract void printArea();  
9  
10 }  
11  
12 class Rectangle extends Shape {  
13     void printArea() {  
14         System.out.println("*** Finding the Area of Rectangle ***");  
15         System.out.print("Enter length and breadth: ");  
16         length = input.nextInt();  
17         breadth = input.nextInt();  
18         System.out.println("The area of Rectangle is: " + length * breadth);  
19     }  
20 }  
21  
22 class Triangle extends Shape {  
23     void printArea() {  
24         System.out.println("\n*** Finding the Area of Triangle ***");  
25         System.out.print("Enter Base And Height: ");  
26         length = input.nextInt();  
27         breadth = input.nextInt();  
28         System.out.println("The area of Triangle is: " + (length * breadth) / 2);  
29     }  
30 }  
31  
32 class Circle extends Shape {  
33     void printArea() {
```

Console Output:

```
<terminated> AbstractClassExample [Java Application] C:\Program Files\Java\jre6\bin\java.exe  
*** Finding the Area of Rectangle ***  
Enter length and breadth: 2 4  
The area of Rectangle is: 8  
  
*** Finding the Area of Triangle ***  
Enter Base And Height: 4 5  
The area of Triangle is: 10  
  
*** Finding the Area of Circle ***  
Enter Radius: 6  
The area of Circle is: 113.04
```

🔔 An abstract class can not be instantiated but can be referenced. That means we can not create an object of an abstract class, but base reference can be created.

In the above example program, the child class objects are created to invoke the overridden abstract method. But we may also create base class reference and assign it with child class instance to invoke the same. The main method of the above program can be written as follows that produce the same output.

Example

```
public static void main(String[] args) {  
    Shape obj = new Rectangle(); //Base class reference to Child class  
    instance  
    obj.printArea();  
}
```

```
        obj = new Triangle();  
        obj.printArea();  
  
        obj = new Cricle();  
        obj.printArea();  
    }
```

8 Rules

An abstract class must follow the below list of rules.

- An abstract class must be created with abstract keyword.
- An abstract class can be created without any abstract method.
- An abstract class may contain abstract methods and non-abstract methods.
- An abstract class may contain final methods that can not be overridden.
- An abstract class may contain static methods, but the abstract method can not be static.
- An abstract class may have a constructor that gets executed when the child class object created.
- An abstract method must be overridden by the child class, otherwise, it must be defined as an abstract class.
- An abstract class can not be instantiated but can be referenced.